

The Alberta Buck - Formal Correctness Proofs (v2)

Rerandomization, Re-Encryption, and Notes Spend Predicates

Perry Kundert

2026-04-22



The Alberta Buck identity system (Identity, Examples) rests on two cryptographic transformations: the *Identity Fountain* blinds a Pointcheval-Sanders credential into a hiding presentation (A, B) that no holder of the identity, the issuer included, can test, and the **approve** handshake re-encrypts the credential's ElGamal ciphertext under the counterparty's public key with a Chaum-Pedersen proof of equality. Everything downstream – bilateral transfers, the BUCK Notes spend circuits (Notes), Identity Guardian authorizations, regulatory recovery – inherits its correctness from these two primitives.

This document states and proves the correctness properties that the rest of the series relies on:

- **Theorem 1:** rerandomization preserves PS signature validity (completeness; a wallet-internal step).
- **Lemma 2:** a uniform random rerandomization scalar yields a signature distributed as a fresh one on the same message. It does not hide the message: a rerandomized pair is a candidate test for anyone holding m , so the wallet never publishes one.
- **Theorem 1':** the published presentation $(A, B) = (a\sigma_1, a\sigma_2 + bY_1)$ satisfies the masked verifier equation (completeness).
- **Theorem 2':** for fresh uniform (a, b) the triple (A, B, P) that the registration transcript reveals is independent of m up to $1/q$ (perfect hiding).
- **Theorems R-HVZK, R-sound, R-comp:** the registration NIZK is honest-verifier zero-knowledge given P , specially sound (extracting a plain PS signature), and composes the credential, ciphertext and key relations on one m .

- **Theorem R-unforge:** registration forgery – by a party with no credential, or with every registration of the target and its identity scalar in hand – reduces to PS unforgeability in the committed-message key form, in the random-oracle model.
- **Theorem R-blind:** the registration transcript gives even the issuer, holding its own key and every certified identity, no advantage beyond DDH in telling which identity registered.
- **Theorem 3:** an honest Chaum-Pedersen prover always verifies (completeness).
- **Theorem 4:** from two accepting transcripts with distinct challenges an efficient extractor recovers the witness (special soundness, 2-extractability).
- **Theorem 5:** the sigma-protocol transcript is perfectly simulatable given only public inputs and the challenge (HVZK).
- **Theorem 6:** combined, an accepted on-chain Chaum-Pedersen proof implies that the re-encrypted ciphertext carries the same issuer-certified identity point $M = m \cdot G$ as the registered credential (end-to-end re-encryption correctness).
- **Theorem 7:** at mint the issuer addresses the recipient’s *receiving key* $pk_{\text{recv}} = kG$, independent of its Identity; the ciphertexts decrypt under k to the intended points, a mint binding forces them over the issuer’s *own* registered Identity (anti-framing), and the issuer learns neither k nor m_{rec} .
- **Theorem 8** (the folded deposit gate, A1/A2): one proof over one witness shows the spender reads the note, that its account decrypts to the Identity m_{rec} , and that a certified accumulator leaf pairs m_{rec} with the reading key – so a thief holding a stolen payload cannot spend, and (A2) the decrypted issuer is registered.
- **Theorem 9** (depositor-binding soundness, B1): an accepted bearer spend proves the depositor’s Identity M_{dep} is registered AND encrypted for the public issuer, who alone names it; a bogus M_{dep} has no certified leaf and reverts.
- **Theorem 10:** the flavor-agnostic nullifier $nf = \text{Poseidon}_3(\rho, idHash, tag)$ deterministically prevents double-spend and, as a PRF over the secret opening, hides the $cm \leftrightarrow nf$ link from any party without the opening (transaction-graph privacy against bulk harvest).
- **Theorem 11:** every accepted spend is mutually decryptable – both counterparties name the other as a *registered* Identity, *enforced* by the membership half (an un-nameable counterparty is un-spendable) – while no third party names either or links mint to spend.
- **Theorem 12** (the A2 key tie): the issuer Identity an accepted A2 deposit certifies is the minter’s own, not a registered sock puppet the same ciphertext also opens to under the recipient’s key – because the note commits the mint binding’s blinded T and the fold opens it, so re-aiming the key needs a discrete logarithm to H_{PED} .

Every theorem is accompanied by an executable `py_ecc` sanity check on BN254. The proofs assume PS unforgeability in the committed-message key form (Pointcheval–Sanders 2016, Section 6.1), the discrete logarithm and decisional Diffie–Hellman assumptions on $\text{alt_bn128 } G_1$, the random oracle model (for the Fiat-Shamir transform and Poseidon hashing), and knowledge soundness of the underlying SNARK proof system (for Theorems 8-9 and 12), with H_{PED} hashed to the curve so that no one knows its logarithm. The presentation results (Theorems 1’, 2’, R-HVZK, R-sound,

R-comp) describe the A' registration implemented on branch `feature/a-prime`; they are stated with proofs here but await independent review, and that branch is not a production release. (PDF, Text)

DRAFT

Contents

1	Preliminaries	6
1.1	Notation	6
1.2	Hardness Assumptions	6
2	Part I: Pointcheval-Sanders Credentials and the Hiding Presentation	8
2.1	Status and Scope of This Part	8
2.2	The Construction	8
2.3	Security Definitions	9
2.4	Theorem 1 (Completeness of Rerandomization)	10
2.5	Theorem 1' (Completeness of the Masked Verifier)	10
2.6	Lemma 2 (Fresh-Signature Distribution for a Fixed Message)	10
2.7	Theorem 2' (Perfect Hiding of the Presentation)	11
2.8	Theorem R-HVZK (Zero-Knowledge of the Registration Transcript)	11
2.9	Theorem R-sound (Special Soundness)	12
2.10	Theorem R-comp (Composition of the Credential, Ciphertext and Key Relations)	12
2.11	Theorem R-unforge (Registration Unforgeability, from PS-CM)	12
2.12	Theorem R-blind (Issuer-Blindness of the Registration Transcript)	13
2.13	Proposition R-fresh (Randomness Reuse Is a Break)	14
2.14	What the Verifier Checks, and Why	14
2.15	Sanity Check (py_ecc on BN254)	15
3	Part II: Chaum-Pedersen Re-Encryption Correctness	18
3.1	Setup	18
3.2	The Statement	18
3.3	Protocol (Fiat-Shamir)	18
3.4	Theorem 3 (Completeness)	19
3.5	Theorem 4 (Special Soundness / 2-Extractability)	19
3.6	Theorem 5 (Honest-Verifier Zero-Knowledge)	20
3.7	Sanity Check (py_ecc on BN254)	20
4	Part III: End-to-End Re-Encryption Correctness	23
4.1	Chain of Trust	23
4.2	Theorem 6 (End-to-End Re-Encryption Correctness)	23
4.3	Why This Suffices for Every Downstream Use Case	23
4.4	The EOA Transfer Mirror (<code>transfer</code> / <code>transferFrom</code>)	24
4.5	Sanity Check (py_ecc on BN254)	24
5	Part IV: BUCK Notes Spend Predicates (A1, A2, B1)	26
5.1	Setup	26
5.2	Theorem 7 (Mint Addressing and Anti-Framing)	27
5.3	Theorem 8 (Folded Deposit Gate – A1, A2)	27
5.4	Theorem 9 (Depositor-Binding Soundness – B1)	29
5.5	Theorem 10 (Nullifier and Double-Spend Prevention)	30
5.6	Theorem 11 (Mutual Decryptability and Transaction-Graph Privacy)	30
5.7	Theorem 12 (The Key Tie – A2 Naming)	31
5.8	Sanity Check (py_ecc on BN254)	31
5.9	Worked Example: An A2 Spend End to End	33

5.10	Mint-Circuit Obligations	33
5.10.1	Obligation 1 (Field-Overflow / Value-Conservation Range Check) – <i>Discharged in circuit</i>	33
5.10.2	Obligation 2 (Single <code>transferFrom</code> $\leftrightarrow$ N Commitments) – <i>Discharged by SNARK binding</i>	34
5.10.3	Obligation 3 (Membership, Nullifier, Value) – <i>Split between mint and spend</i>	34
6	Security Assumptions Summary	35
7	Implementation Notes	36
7.1	What the Contract Must Enforce	36
7.2	What the Prover Must Enforce	36
7.3	Scope Not Covered Here	36
7.4	The Long-Horizon Threat: Harvest-Now, Decrypt-Later	37
8	Review Status and Remaining Work for Part I	39
8.1	What the Reviewer Is Asked to Check	39
8.2	What Would Strengthen the Argument	40

DRAFT

1 Preliminaries

1.1 Notation

Symbol	Meaning
q	Prime order of G_1, G_2, G_T on BN254
G, g_2	Generators of G_1 and G_2
$e : G_1 \times G_2 \rightarrow G_T$	Bilinear, non-degenerate pairing (type-3)
O	Point at infinity (group identity)
$\mathbb{Z}_q^*$	Non-zero scalars mod q
$(x, y)/(X, Y, Y_1)$	Issuer secret / public key: $X = xg_2, Y = yg_2, Y_1 = yG$; the registry checks $e(Y_1, g_2) = e(G, Y)$ at trust time
$m \in \mathbb{Z}_q$	Identity scalar: $m = \text{keccak}(\tau_{\text{ID}} \parallel \text{identity_data}) \bmod q$ ¹
$M = m \cdot G \in G_1$	Identity point: the value every ElGamal ciphertext encrypts ²
$\sigma = (\sigma_1, \sigma_2) \in G_1^2$	PS signature: $\sigma_1 = h, \sigma_2 = (x + my)h$ ³
(sk, pk)	Identity key pair on G_1 : $pk = sk \cdot G$
$E = (R, C) \in G_1^2$	ElGamal ciphertext: $R = rG, C = M + r \cdot pk$
$H(\cdot)$	Cryptographic hash modelled as a random oracle (Fiat-Shamir); every transcript ends in its protocol's v2 domain word ke
$H_{\text{PED}} \in G_1$	Pedersen generator hashed to the curve under AlbertaBuck/Pedersen/H/v2 : no one knows $\log_G H_{\text{PED}}$. Part IV writes it H
(k, pk_{recv})	Receiving secret and key, $pk_{\text{recv}} = kG$: what addressed notes are encrypted to, derived from wallet seed material and inde
$a, b \in \mathbb{Z}_q^*$	Presentation scalars, fresh per account: a rerandomizes, b blinds on the Y base
$(A, B) \in G_1^2$	The published presentation $A = a\sigma_1, B = a\sigma_2 + bY_1 = (x + my)A + bY_1$
$P = mA + bG$	The one credential-side point derivable from a registration transcript; $B = xA + yP$
m_t, b_t, r_t, sk_t	Registration NIZK nonces; $C_1 = m_t A + b_t G, T_C, T_R, T_{key}$ its commitments; s_m, s_b, s_r, s_{sk} its responses

Response convention: Part I-bis (Theorems R-*) writes responses as $s = w_t + ew$, matching the wallet and the Solidity verifier; Part II writes the Chaum-Pedersen responses as $s = k - ew$. The two are equivalent up to the sign of e in the verification equations. Part IV reuses b and H with their own meanings (B1's blind, and H_{PED}); the presentation's b and P are local to Part I. Where e appears as a Fiat-Shamir challenge it is a scalar; as $e(\cdot, \cdot)$ it is the pairing.

All group operations in this document are written additively. BN254 has cofactor 1 on G_1 , so every non-identity $P \in G_1$ has order q ; there are no small-subgroup elements.

1.2 Hardness Assumptions

Five assumptions underpin the theorems in this document. Four are already required by Ethereum's own security model or by any pairing-based / zk-SNARK construction built on the **ecPairing** precompile; the PS assumption is specific to the credential scheme and, since the issuer key now includes yG , it is the committed-message form of that assumption.

1. **Discrete Logarithm (DLog) on G_1** : given $P, Q \in G_1$, finding $\alpha \in \mathbb{Z}_q$ with $Q = \alpha P$ is infeasible. Protects ElGamal IND-CPA security, Schnorr unforgeability, and Chaum-Pedersen soundness, and keeps $\log_G H_{\text{PED}}$ unknown, which Part IV's commitments rest on.
2. **Decisional Diffie-Hellman (DDH) on G_1** : distinguishing $(G, \alpha G, \beta G, \alpha \beta G)$ from a random tuple is infeasible. Protects ElGamal plaintext hiding, which is the only computational lock left on a registration after Theorem 2' (the issuer-blindness theorem of the main paper and the long-horizon section below).
3. **PS unforgeability, committed-message key form (PS-CM)**: the Pointcheval-Sanders single-message scheme is EUF-CMA when the public key also exposes $Y_1 = yG$ in G_1 . Section

¹Disclosed to counterparties by design, so nothing is encrypted under $M = mG$; insufficient for registration or spend authorization. $\tau_{\text{ID}} = \text{AlbertaBuck/Identity/Scalar/v2}$

²Public (recoverable from m); the KYC-registered anchor

³Write-capability secret: authorizes account registration. Held only by the Identity owner; never disclosed to counterparties and never published: the holder publishes the presentation (A, B)

6.1 of their 2016 paper⁴ publishes exactly $(G, Y_1; g_2, X, Y)$ and lets the signer sign commitments $tG + mY_1$, which the user unblinds to $(h, (x + my)h)$; a plain-message signing oracle is the special case $t = 0$, so an adversary against our scheme is weaker than the Section 6.1 adversary, whose scheme is proved EUF-CMA under that paper's Assumption 2, an interactive assumption in type-3 groups. Theorem R-unforge reduces registration forgery to it directly. The issuer keeps the signature base h with unknown discrete logarithm, which `ps_sign` does by sampling it. Nothing in this document depends on q-SDH.

4. **Random Oracle Model (ROM):** the Fiat-Shamir transform is applied to sigma protocols whose security is proved in the ROM, with the hash function H modelled as a random oracle bound to all public inputs and context (caller address, chain id, etc.). Poseidon (commitment hash, accumulator leaves and nullifier PRF) and the hash-to-curve that derives H_{PED} are also modelled as random oracles for the BUCK Notes proofs in Part IV. Each transcript's domain word keeps the oracles of different protocols apart.
5. **Knowledge soundness of the underlying zk-SNARK proof system** (Part IV only): the SNARK system used for BUCK Notes spend circuits (Groth16, PLONK, or any knowledge-sound construction over BN254) admits a polynomial-time witness extractor on every accepting proof. This is a system-specific assumption depending on the chosen scheme's trusted setup or universal setup.

In Part I, Theorems 1 and 1' follow from bilinearity alone, Lemma 2 and Theorem 2' are information-theoretic, and Theorem R-HVZK is a perfect simulation (the NIZK invokes the ROM); Theorem R-sound's extractor is unconditional; Theorem R-unforge is computational (PS-CM, ROM) and Theorem R-blind is computational (DDH, ROM). No assumption is made about the issuer beyond honest issuance: it holds (x, y) and can sign at will, so unforgeability is stated against every party except the issuer, and issuer-blindness against the issuer. The Chaum-Pedersen proofs (Part II) are interactive-sigma-protocol facts; the NIZK corollary (Part II, end) invokes the ROM. Part IV (Notes spend predicates) additionally invokes assumption (5) – knowledge soundness of the underlying SNARK system – to lift the sigma-protocol guarantees of Theorems 4-5 into the SNARK setting.

⁴Pointcheval, D. and Sanders, O. "Short Randomizable Signatures." CT-RSA 2016, LNCS 9610, pp. 111-126. The single-message scheme is proved EUF-CMA under that paper's Assumption 1, an interactive LRSW-style assumption in type-3 bilinear groups (an earlier edition of this document attributed it to q-SDH; that was wrong). Section 6.1 (signatures on committed messages) publishes yG in G_1 as the A' key does and proves security under the stronger Assumption 2. Pointcheval, D. and Sanders, O. "Reassessing Security of Randomizable Signatures." CT-RSA 2018, gives non-interactive q-type formulations for modified variants; whether the key form used here is covered by one of them is part of the phase-6 review.

2 Part I: Pointcheval-Sanders Credentials and the Hiding Presentation

2.1 Status and Scope of This Part

The registration presentation proved here (the "A'" design) is implemented on branch `feature/a-prime` across the Python executable specification (`alberta_buck/wallet/ps.py`, `nizk.py`), the Solidity verifier (`src/IdentityRegistry.sol: trustIssuer, _verifyCredential, _fsRegister, _checkPresentation`), the Rust kernel (`core/rust/buck-identity/src/{ps,nizk}.rs`) with its PyO3 and wasm bindings, and the JavaScript wrapper (`core/js/src/identity-core.js`), all checked against one set of vectors. This part is the security argument for it, written in full so that an independent reviewer can confirm or refute each step; the review itself has not happened, and until it has every result below is a claim of that branch, not a property of the system. Section 8 lists what the reviewer is asked to check and what would strengthen the argument further.

What is claimed. A holder of a PS credential on an identity scalar m registers an account by publishing a blinded pair (A, B) and a Schnorr-family proof. The pair and the proof together reveal nothing about m beyond what the account's ElGamal ciphertext reveals: unconditionally on the credential side (Theorems 2', R-HVZK) and under DDH on the ciphertext side (Theorem R-blind). An accepting registration implies possession of a credential on the encrypted m , and neither a party who has seen any number of the holder's registrations nor one who has learned m can produce one: registration forgery reduces to PS unforgeability in the committed-message key form, in the random-oracle model, with the usual forking loss (Theorems R-sound, R-unforge). Fresh randomness per registration is security critical (Proposition R-fresh).

What is not covered here: the identity-derived A1/A2 Note receiving keys and the issuance, policy and accumulator metadata of the review's findings R5 and R6; duplicate issuance by a corrupt issuer and the optional holder secret; wallet-side leakage (RNG quality, side channels, backup formats that might export σ , any of which reintroduces credential harvesting); and post-quantum security, for which see the long-horizon section.

2.2 The Construction

A Pointcheval-Sanders signature on $m \in \mathbb{Z}_q$ is a pair $\sigma = (\sigma_1, \sigma_2) \in G_1^2$ with $\sigma_1 \neq O$ satisfying the pairing equation

$$e(\sigma_1, X + m \cdot Y) = e(\sigma_2, g_2).$$

Keys and issuance. The issuer holds (x, y) and publishes

$$X = x \cdot g_2, \quad Y = y \cdot g_2, \quad Y_1 = y \cdot G.$$

The registry accepts a key only if $Y_1 \neq O$ and $e(Y_1, g_2) = e(G, Y)$ (`trustIssuer`). Issuance is the plain scheme's: on m the issuer samples $h = t \cdot G$ for uniform nonzero t it does not disclose and returns $\sigma = (h, (x + my)h)$. The credential never leaves the wallet. *Rerandomization*, $\sigma' = (t \cdot \sigma_1, t \cdot \sigma_2)$ for $t \in \mathbb{Z}_q^*$, is a wallet-internal step.

Presentation. For each account the wallet samples independent uniform nonzero a, b, sk, r and publishes

$$A = a \cdot \sigma_1, \quad B = a \cdot \sigma_2 + b \cdot Y_1 = (x + my) \cdot A + b \cdot Y_1, \quad pk = sk \cdot G, \quad E = (R, C) = (rG, mG + r \cdot pk),$$

and discards a, b after proving. Write $P = mA + bG$; then $B = xA + yP$. P is the one credential-side value a verifier can derive from the transcript, and the results below are stated in terms of it.

Registration proof. Witness (m, b, r, sk) ; nonces (m_t, b_t, r_t, sk_t) uniform in $\mathbb{Z}_q$; context $\text{ctx} = (\text{registrant}, \text{chainid}, \text{registry}, \text{DOMAIN})$ with $\text{DOMAIN} = \text{keccak}(\text{AlbertaBuck/FiatShamir/IdentityRegistry/R})$.

$$\begin{aligned} C_1 &= m_t A + b_t G, & T_C &= m_t G + r_t pk, & T_R &= r_t G, & T_{key} &= sk_t G, \\ e &= H(A, B, R, C, pk, C_1, T_C, T_R, T_{key}, \text{ctx}) \bmod q, \\ s_m &= m_t + em, & s_b &= b_t + eb, & s_r &= r_t + er, & s_{sk} &= sk_t + e sk. \end{aligned}$$

Published: $(A, B), pk, E, (e, s_m, s_b, s_r, s_{sk}, C_1, T_C, T_R, T_{key})$, plus the issuer's address, which selects the key. Credential-based contract binding uses the identical proof with the target contract as registrant.

Verifier.

- (g) $A, B, pk, R \neq O$; all points on the curve; $e, s_m, s_b, s_r, s_{sk} < q$; registrant unregistered; issuer trusted
- (fs) $e = H(A, B, R, C, pk, C_1, T_C, T_R, T_{key}, \text{ctx}) \bmod q$
- (b) $s_m G + s_r pk = eC + T_C$
- (c) $s_r G = eR + T_R$
- (k) $s_{sk} G = T_{key} + e pk$
- (a') $e(s_m A + s_b G - C_1, Y) \cdot e(eA, X) \cdot e(-eB, g_2) = 1$

One commitment C_1 carries both credential exponents; the nonce m_t is shared between C_1 and T_C , and r_t between T_C and T_R .

2.3 Security Definitions

Three games. Advantage is $|\Pr[\text{guess} = \beta] - 1/2|$ for the distinguishing games and the success probability for the forgery game.

Presentation hiding (HIDE). The adversary chooses the issuer key (x, y) – it may be the issuer – two scalars m_0, m_1 and valid credentials σ^0, σ^1 on them. The challenger samples uniform nonzero a, b and a bit β and returns $(A, B) = (a\sigma_1^\beta, a\sigma_2^\beta + bY_1)$ together with $P = m_\beta A + bG$. The adversary guesses β . Returning P makes the game at least as strong as anything the real transcript reveals (Theorem R-HVZK's remark).

Registration unforgeability (RU). The challenger samples (x, y) and publishes (X, Y, Y_1) . The adversary has a random oracle and two oracles: **Sign**(m) returns a credential on m of its choice, and the set of queried messages is Q ; **Present**(m, ctx), for *any* m including ones not in Q , returns a complete honest registration transcript (pk, E, A, B, π) of m with fresh (a, b, sk, r) and the given context, and the adversary is also given m itself. This models a party who has seen the target's registrations and has learned the target's identity scalar, as every counterparty does. The adversary wins if it outputs an accepting transcript whose extracted witness (Theorem R-sound) has $m^* \notin Q$; m^* may well be a message it obtained presentations of, since only credentials count as queries. One game therefore covers both "no credential on the encrypted identity" (soundness) and the review's R3 (harvesting).

Issuer-blindness (IB). The adversary is the issuer: it holds (x, y) and chooses two certified identities m_0, m_1 with their credentials. The challenger registers one at random with fresh (a, b, sk, r)

and honest randomness and returns the full public transcript (pk, E, A, B, π) with a context of the adversary's choice. Many registrations are handled by a hybrid over them.

2.4 Theorem 1 (Completeness of Rerandomization)

Statement. For every valid PS signature σ on m under issuer key (X, Y) , and for every $t \in \mathbb{Z}_q^*$, the pair $\sigma' = (t\sigma_1, t\sigma_2)$ is also a valid PS signature on m under the same issuer key.

Proof. By bilinearity of e :

$$\begin{aligned}
e(\sigma'_1, X + mY) &= e(t\sigma_1, X + mY) && \text{(definition)} \\
&= e(\sigma_1, X + mY)^t && \text{(bilinearity)} \\
&= e(\sigma_2, g_2)^t && \text{(original validity)} \\
&= e(t\sigma_2, g_2) && \text{(bilinearity)} \\
&= e(\sigma'_2, g_2).
\end{aligned}$$

Non-degeneracy: G_1 has prime order q , so $t\sigma_1 = O$ iff $t \equiv 0 \pmod{q}$ or $\sigma_1 = O$. Both are excluded by hypothesis. ■

Scope. The wallet uses this step to re-issue the credential to itself; nothing produced by it is published. The published object is the presentation, whose completeness is Theorem 1'.

2.5 Theorem 1' (Completeness of the Masked Verifier)

Statement. For every valid credential, every $a, b \in \mathbb{Z}_q^*$ and every sk, r , the honest transcript is accepted; in particular the presentation satisfies

$$e(B, g_2) = e(A, X) \cdot e(mA + bG, Y),$$

equivalently $B = xA + yP$.

Proof. Split B :

$$\begin{aligned}
e(a\sigma_2, g_2) &= e(\sigma_2, g_2)^a = e(\sigma_1, X + mY)^a = e(A, X + mY) = e(A, X) \cdot e(mA, Y), \\
e(bY_1, g_2) &= e(byG, g_2) = e(bG, g_2)^y = e(bG, Y).
\end{aligned}$$

Multiplying, $e(B, g_2) = e(A, X) \cdot e(mA + bG, Y)$. Then $s_m A + s_b G - C_1 = e(mA + bG) = eP$, so (a') is $[e(P, Y) e(A, X) / e(B, g_2)]^e = 1$; (b), (c), (k) are the Schnorr identities; and $A \neq O$ since $\sigma_1 \neq O$ and $a \neq 0$. ■

Implementation guard. With $A = O$ the masked equation reads $e(B, g_2) = e(bG, Y)$, which $B = bY_1$ satisfies *for every* m : the credential term vanishes and the relation says nothing about the identity. The registration contract MUST reject $A = O$ (and $B = O$). This is the same degenerate case that the earlier published form met as $\sigma'_1 = O$ (from $t = 0$); under the presentation it is $a = 0$. Theorem 1' depends on $a \neq 0$; the contract's guard is the runtime witness of that hypothesis, and it is security critical rather than hygenic.

2.6 Lemma 2 (Fresh-Signature Distribution for a Fixed Message)

Statement. Let σ be any fixed valid PS signature on m . If t is sampled uniformly from $\mathbb{Z}_q^*$, then the distribution of $\sigma' = (t\sigma_1, t\sigma_2)$ is identical to the distribution of $\text{PS.Sign}(sk_{\text{issuer}}, m)$ for h sampled uniformly from $G_1 \setminus \{O\}$, and is *independent* of σ .

Proof. Since G_1 is cyclic of prime order q and $\sigma_1 \neq O$, the map $\varphi_\sigma : \mathbb{Z}_q^* \rightarrow G_1 \setminus \{O\}$, $t \mapsto t\sigma_1$ is a bijection. Hence σ'_1 is uniform on $G_1 \setminus \{O\}$ regardless of σ_1 . Given $\sigma'_1 = h'$, the second component is forced: $\sigma'_2 = (x + my)h'$, exactly what a fresh PS.Sign with random h' would output. ■

What this lemma does not say. The rerandomized pair is a valid signature on m : any party holding a candidate m_i decides whether $e(\sigma'_1, X + m_i Y) = e(\sigma'_2, g_2)$. The issuer holds every m it has certified, and every counterparty learns m by design, so a *published* rerandomized pair is a candidate test on the identity, not an unlinkable credential (review finding R1 in doc/review/identity-findings-2.org). The lemma is a statement about the wallet's internal step; the privacy property of the published object is Theorem 2'. Earlier versions of this document called this result "statistical unlinkability" and drew the issuer-blindness of the identity layer from it; that reading is withdrawn.

2.7 Theorem 2' (Perfect Hiding of the Presentation)

Statement. Every adversary in HIDE, computationally unbounded and holding the issuer key, has advantage at most $1/q$. More precisely: for a fixed credential on m and independent uniform $a, b \in \mathbb{Z}_q^*$, the distribution of (A, B, P) is within statistical distance $1/q$ of $(U_1, xU_1 + yU_2, U_2)$, where U_1 is uniform on $G_1 \setminus \{O\}$, U_2 is uniform on G_1 , and the two are independent; this reference distribution does not depend on m .

Proof. Fix β . $A = a\sigma_1^\beta$ is uniform on $G_1 \setminus \{O\}$ by the bijection of Lemma 2. Conditioned on A , the map $b \mapsto m_\beta A + bG$ is a bijection $\mathbb{Z}_q \rightarrow G_1$, so for b uniform on $\mathbb{Z}_q^*$ the point P is uniform on $G_1 \setminus \{m_\beta A\}$, independent of A . By Theorem 1', $B = xA + yP$ is a function of (A, P) . The distributions for $\beta = 0$ and $\beta = 1$ therefore differ only in which single point P avoids, at statistical distance at most $1/q$, and nothing else the challenger outputs depends on β . ■

Remarks. (i) The adversary knows x, y : the result is unconditional and holds against the issuer. (ii) Allowing $b = 0$ would make the distribution exactly independent of m , but $b = 0$ publishes a rerandomized signature, which Lemma 2's remark shows is testable; excluding it costs $1/q$. (iii) For every candidate m_i the point $P - m_i A$ is a well-formed $b_i G$, so no candidate is distinguished by any relation among public values. (iv) Whatever the registration transcript reveals about m beyond (A, B, P) is what the ElGamal ciphertext (pk, R, C) reveals (Theorem R-HVZK), which is the plaintext-checking problem under DDH (Theorem R-blind).

2.8 Theorem R-HVZK (Zero-Knowledge of the Registration Transcript)

Statement. Given $(A, B, P, pk, R, C, \text{ctx})$ and a challenge e , the simulator draws s_m, s_b, s_r, s_{sk} uniform on $\mathbb{Z}_q$ and sets

$$C_1 = s_m A + s_b G - eP, \quad T_C = s_m G + s_r pk - eC, \quad T_R = s_r G - eR, \quad T_{key} = s_{sk} G - e pk.$$

The simulated transcript is distributed identically to an honest transcript with the same challenge. In the random-oracle model the NIZK is zero-knowledge: the simulator programs H at the transcript to e , aborting if that input was queried before, which happens with probability at most q_H/q over the adversary's q_H oracle queries because C_1 is uniform and fresh.

Proof. In an honest transcript with fixed witness and challenge, each response is a uniform nonce plus a constant, so the four responses are uniform and independent, and the four commitments are the unique points satisfying the verification equations for those responses. The simulator samples responses from the same distribution and solves the same equations. ■

Remark (what the simulator needs, and what the old proof leaked). The simulator needs P , which is public: $P = e^{-1}(s_m A + s_b G - C_1)$. The transcript is therefore a deterministic function of $(A, B, P, pk, E, \text{ctx})$ and simulator randomness, and Theorem 2' says (A, B, P) is independent of m . The pre-A' proof published a separate commitment $A_{ps} = m_t A$ with the response s_m , from which any observer computes $mA = e^{-1}(s_m A - A_{ps})$, a candidate test on m needing neither the second signature point nor any issuer key (review finding R2). Folding $b_t G$ into the same commitment is what removes the test.

2.9 Theorem R-sound (Special Soundness)

Statement. From two accepting transcripts with the same $(A, B, pk, R, C, C_1, T_C, T_R, T_{key}, \text{ctx})$ and challenges $e \neq e'$, one computes

$$m^* = \frac{s_m - s'_m}{e - e'}, \quad b^* = \frac{s_b - s'_b}{e - e'}, \quad r^* = \frac{s_r - s'_r}{e - e'}, \quad sk^* = \frac{s_{sk} - s'_{sk}}{e - e'}$$

such that $pk = sk^* G$, $(R, C) = (r^* G, m^* G + r^* pk)$, and $e(A, X + m^* Y) = e(B - b^* Y_1, g_2)$. Hence $(A, B - b^* Y_1)$ is a plain PS signature on the m^* encrypted in E , under a key whose secret is sk^* .

Proof. Dividing the two instances of (a') and raising to the power $(e - e')^{-1}$: $e(m^* A + b^* G, Y) e(A, X) = e(B, g_2)$, i.e. $e(A, X + m^* Y) e(b^* G, Y) = e(B, g_2)$. Since $Y_1 = yG$ (enforced at `trustIssuer`), $e(b^* G, Y) = e(b^* Y_1, g_2)$, which rearranges to the stated equation; $A \neq O$ by (g). Relations (b), (c), (k) extract r^*, sk^* and the same m^* exactly as in Theorem 4 of Part II. ■

This is the only place Y_1 enters the argument; the verifier never multiplies by it.

2.10 Theorem R-comp (Composition of the Credential, Ciphertext and Key Relations)

Statement. Relations (a'), (b), (c), (k) under one challenge form a sigma protocol for the conjunction

$$\exists(m, b, r, sk) : \quad B = xA + y(mA + bG), \quad (R, C) = (rG, mG + rpk), \quad pk = sk G,$$

with special soundness and honest-verifier zero-knowledge. In particular an accepting registration proves that the *same* m is certified by the presentation and encrypted in E , and that the registrant holds sk for pk .

Proof. AND-composition of sigma protocols under a common challenge preserves both properties. The shared nonce m_t makes the m extracted from (a') and from (b) the same scalar $(s_m - s'_m)/(e - e')$; the shared r_t does the same for r between (b) and (c). The simulator of Theorem R-HVZK already simulates all four commitments from the same responses. ■

2.11 Theorem R-unforge (Registration Unforgeability, from PS-CM)

Statement. Let $\mathcal{A}$ be an adversary in RU making q_H random-oracle queries and q_P presentation queries, whose accepting output has extracted $m^* \notin Q$ with probability ε . There is a PS-CM forger $\mathcal{B}$ with success probability at least $\varepsilon^2/q_H - (q_P + q_H + 1)/q$, up to the forking lemma's constants.

Proof. $\mathcal{B}$ receives the PS-CM public key (X, Y, Y_1) and a signing oracle $\mathcal{S}$, and forwards the key to $\mathcal{A}$.

Sign queries. Forwarded to $\mathcal{S}$; $\mathcal{B}$ records Q .

Presentation queries. $\mathcal{B}$ must answer $\text{Present}(m, \text{ctx})$ for identities it holds no credential on. It queries $\mathcal{S}$ once, on a message m_d chosen uniformly at random, obtaining $\sigma_d = (h_d, (x + m_d y)h_d)$. For each query it samples fresh a, b, sk, r and sets

$$A = ah_d, \quad B = a\sigma_{d,2} + bY_1 = xA + y(m_d A + bG), \quad P := m_d A + bG, \quad pk = skG, \quad E = (rG, mG + rpk),$$

using the real m , which the adversary holds anyway, and simulates π with the simulator of Theorem R-HVZK, programming H at the transcript.

Forgery. When $\mathcal{A}$ outputs an accepting transcript whose challenge was the i -th oracle query, $\mathcal{B}$ rewinds $\mathcal{A}$ to that query with a fresh answer (the forking lemma⁵); with probability about ε^2/q_H it obtains a second accepting transcript with the same commitments and a different challenge, extracts (m^*, b^*) by Theorem R-sound, and outputs $(m^*, (A^*, B^* - b^*Y_1))$.

Indistinguishability of the simulation. In a real Present answer, A is uniform nonzero, P is uniform on $G_1 \setminus \{mA\}$, and $B = xA + yP$. In the simulation A is again uniform nonzero (a rerandomization of h_d), P is uniform on $G_1 \setminus \{m_d A\}$, and $B = xA + yP$. The two differ by at most $1/q$ per query (Theorem 2'). The ciphertext is formed identically. The proof is simulated, which by Theorem R-HVZK is exact given $(A, B, P, pk, E, \text{ctx})$ up to the programming abort. The relation " $P = mA + bG$ for the m inside E " is not checkable by anyone without b , since every m admits some b . So nothing in $\mathcal{A}$'s view distinguishes the simulated world from the real one beyond $q_P/q + q_H/q$.

Validity of the forgery. By Theorem R-sound the pair verifies as a PS signature on m^* . $\mathcal{A}$ wins only if $m^* \notin Q$. The one message $\mathcal{B}$ queried that $\mathcal{A}$ does not see is m_d ; since $\mathcal{A}$'s view is independent of m_d up to the statistical distance above, $\Pr[m^* = m_d] \leq (1 + q_P)/q$. Hence the output is a forgery on a message never queried to $\mathcal{S}$. ■

What this closes. A party that has learned m from a receipt and holds every registration of the target (review finding R3); a party that copies a registration to another address or chain (ctx is in the transcript, and a fresh proof needs sk and b anyway); and a party with no credential at all. Identity disclosure therefore stops conferring registration authority under an honest issuer. The observation that stripping B needs $bY_1 = y(bG)$ from bG and Y_1 , a CDH instance, remains true but is not load-bearing.

What it does not close. The issuer, who can sign. Two credentials on the same identity issued to different people are a certification-policy problem, not a cryptographic one.

2.12 Theorem R-blind (Issuer-Blindness of the Registration Transcript)

Statement. Every PPT adversary in IB has advantage at most $\text{Adv}_{\text{DDH}} + 1/q + q_H/q$ per registration. This is the credential lock the main paper's issuer-blindness theorem needs; its plaintext lock is the DDH term.

Proof (hybrids). H_0 : the real game with bit β . H_1 : replace π by the simulated proof from $(A, B, P, pk, E, \text{ctx})$; identical up to the programming abort (Theorem R-HVZK). H_2 : replace (A, B, P) by $(U_1, xU_1 + yU_2, U_2)$ with U_1 uniform nonzero and U_2 uniform, at statistical distance at most $1/q$ (Theorem 2'), computable by the challenger since the adversary supplied (x, y) . In H_2 the only object that depends on β is $E = (rG, m_\beta G + rpk)$ with $pk = skG$ for secret sk, r :

⁵Pointcheval, D. and Stern, J. "Security Arguments for Digital Signatures and Blind Signatures." Journal of Cryptology 13(3),

1. The forking lemma: an adversary that convinces a sigma-protocol verifier with non-negligible probability can be rewound to produce two accepting transcripts with distinct challenges, from which the witness is extracted.

the ElGamal encryption of a known plaintext under a fresh key, whose indistinguishability is the plaintext-checking problem, i.e. deciding whether $(G, pk, R, C - m_\beta G)$ is a DDH tuple. ■

Scope. The theorem is about the registration transcript. It says nothing about the issuer's address (which identifies the anonymity set), about timing or gas, or about identity-derived Note keys.

2.13 Proposition R-fresh (Randomness Reuse Is a Break)

Each of the following is an explicit break if the wallet reuses a value:

- b reused across two registrations of one identity: $P_1 - P_2 = m(A_1 - A_2)$, so a holder of a candidate m_i tests $P_1 - P_2 - m_i(A_1 - A_2) = O$. Links the accounts and identifies the identity (sanity check, step 8).
- a reused with the same credential: $A_1 = A_2$; links the accounts.
- m_t reused across two proofs: $s_m - s'_m = (e - e')m$ reveals m to everyone.
- b_t reused: reveals b , after which $B - bY_1$ is a verifiable signature on m , i.e. the credential is published.
- r_t or sk_t reused: reveal r (hence M from $C - rpk$) and sk .

The wallet draws all six from the platform CSPRNG (`rand_scalar`). Deriving the four nonces deterministically from the witness and the transcript, in the style of RFC 6979, would remove the RNG from the trusted base for m_t, b_t, r_t, sk_t ; a and b must still be fresh per registration.

2.14 What the Verifier Checks, and Why

Each item is present in `src/IdentityRegistry.sol` with the revert string given.

- $A \neq O$ (A=0; critical). With $A = O$ the term $e(eA, X)$ vanishes and (a') reduces to $e(s_b G - C_1, Y) = e(eB, g_2)$, which $B = bY_1$, $C_1 = b_t G$ satisfy for every m : a registration with no credential. Reachable by choosing $a = 0$.
- $B \neq O$ (B=0; defensive). $B = O$ means $xA = -yP$; producing such a pair needs x/y . Rejected so that no degenerate encoding is accepted.
- $pk \neq O$ (pk=0; critical). Relation (k) is vacuous at $pk = O$: no account-key ownership.
- $R \neq O$ (R=0; critical). $r = 0$ makes $C = mG$, publishing the identity point.
- Canonical scalars (`bad scalar`). Two encodings of one proof would otherwise be accepted; hygiene, since `ctx` already prevents replay.
- On-curve and subgroup. G_1 has prime order on BN254, so on-curve suffices for G_1 inputs; the pairing precompile rejects G_2 points outside the subgroup, and X, Y are governance-set.
- Y_1 consistency at `trustIssuer` (Y1=0, Y1 inconsistent). Theorem R-sound's last step needs $Y_1 = yG$. A wrong Y_1 cannot help a forger, because the verifier never multiplies by Y_1 ; it would only make honest presentations fail, which the check prevents.

- Fiat-Shamir coverage (**bad FS challenge**). The transcript hashes every public point of the statement, all four commitments, the registrant, the chain id, the registry address and the versioned domain. The issuer address is *not* hashed: the proof is bound to the issuer's key through equation (a') itself, so it cannot verify under a different key, and two issuers publishing the same key are indistinguishable by construction.
- Registrant = `msg.sender` (or the target contract for binding). The proof is useless to any other address.
- Nonzero caller-supplied accumulator leaves are rejected (**unchecked identity leaf**); see the accumulator note in Part IV.
- Storage, not calldata, for later use of E and pk . Downstream proofs (approve, spend) read the registered values.

2.15 Sanity Check (py_ecc on BN254)

The block keeps m, x, y, X, Y, G_1, G_2 and `rand` defined for the later blocks. It samples h as a known multiple of G so that step 3 can exhibit the second opening of Theorem 2'; a real issuer never reveals that logarithm. Step 7 uses SHA-256 as a stand-in random oracle; the point is only that the simulated challenge is not the hash of the simulated commitments.

```
import secrets, hashlib
from py_ecc.bn128 import bn128_curve as bc
from py_ecc.bn128 import pairing, FQ12

ORDER = bc.curve_order
G1, G2 = bc.G1, bc.G2
multiply, add, neg, eq = bc.multiply, bc.add, bc.neg, bc.eq

def rand():
    return secrets.randbelow(ORDER - 1) + 1

# Issuer key pair: (X, Y) in G2 as before, plus the G1 image Y1 = y G
x, y = rand(), rand()
X, Y = multiply(G2, x), multiply(G2, y)
Y1 = multiply(G1, y)

# Alice's identity and PS credential sigma = (h, (x + m y) h). This check
# knows log_G h (it samples h as a multiple of G) and uses it only for the
# perfect-hiding witness in step 3; a real issuer keeps that log secret.
m = int(hashlib.sha256(b"Alice Johnson | Alberta | AIC-2026").hexdigest(), 16) % ORDER
log_h = rand()
h = multiply(G1, log_h)
sigma = (h, multiply(h, (x + m * y) % ORDER))

def present(sigma, a, b):
    # (A, B) = (a sigma_1, a sigma_2 + b Y1)
    return (multiply(sigma[0], a), add(multiply(sigma[1], a), multiply(Y1, b)))

def masked_verify(A, B, m, b):
    # Theorem 1': e(B, g2) == e(A, X) e(mA + bG, Y)
    P = add(multiply(A, m), multiply(G1, b))
    return pairing(G2, B) == pairing(X, A) * pairing(Y, P)

def raw_verify(S1, S2, m):
    # the pre-A' candidate test: e(S1, X + mY) == e(S2, g2)
    return pairing(add(X, multiply(Y, m)), S1) == pairing(G2, S2)

# --- 1. Theorem 1': the honest presentation satisfies the masked equation ---
a, b = rand(), rand()
A, B = present(sigma, a, b)
assert not eq(A, bc.Z1) and not eq(B, bc.Z1)
assert masked_verify(A, B, m, b), "Theorem 1' FAILED"
```

```

print("Theorem 1' (masked completeness):    PASS")

# --- 2. Lemma 2 and R1: the raw test identifies t*sigma but not (A, B) ---
t = rand()
sigma_p = (multiply(sigma[0], t), multiply(sigma[1], t))
assert raw_verify(sigma_p[0], sigma_p[1], m), "Lemma 2: rerandomized pair must verify"
assert not raw_verify(A, B, m), "R1: the presentation must NOT verify as a signature on m"
print("Lemma 2 + R1 (raw candidate test):    PASS (t*sigma identified; (A, B) not)")

# --- 3. Theorem 2': another identity m' opens the SAME (A, B, P) ---
m_prime = rand()
log_A = (a * log_h) % ORDER
b_prime = (b + (m - m_prime) * log_A) % ORDER
P = add(multiply(A, m), multiply(G1, b))
assert eq(P, add(multiply(A, m_prime), multiply(G1, b_prime))), "Theorem 2': P differs"
assert masked_verify(A, B, m_prime, b_prime), "Theorem 2': (A, B) must verify for m' too"
print("Theorem 2' (perfect hiding):          PASS (same (A, B, P) opens to m and m')")

# --- 4. Theorem R-sound: two transcripts sharing C1 extract (m*, b*) ---
m_t, b_t = rand(), rand()
C1 = add(multiply(A, m_t), multiply(G1, b_t))
e1, e2 = rand(), rand()
s_m1, s_b1 = (m_t + e1 * m) % ORDER, (b_t + e1 * b) % ORDER
s_m2, s_b2 = (m_t + e2 * m) % ORDER, (b_t + e2 * b) % ORDER
inv = pow((e1 - e2) % ORDER, -1, ORDER)
m_star, b_star = ((s_m1 - s_m2) * inv) % ORDER, ((s_b1 - s_b2) * inv) % ORDER
assert m_star == m and b_star == b
assert raw_verify(A, add(B, neg(multiply(Y1, b_star))), m_star), "R-sound: not a PS signature"
print("Theorem R-sound (extraction):          PASS ((A, B - b* Y1) is a PS signature on m*)")

# --- 5. The guard: with A = 0 the masked equation holds for EVERY m ---
AO, BO = bc.Z1, multiply(Y1, b)          # what a = 0 would publish
assert masked_verify(AO, BO, m, b) and masked_verify(AO, BO, m_prime, b)
assert eq(AO, bc.Z1)                    # ... which is why the contract rejects A = 0
print("Guard (A != 0):                      PASS (A = 0 verifies for every m; rejected)")

# --- 6. Theorem R-HVZK: the simulator needs only (A, B, P) and e ---
e_s, s_m_s, s_b_s = rand(), rand()
C1_s = add(add(multiply(A, s_m_s), multiply(G1, s_b_s)), neg(multiply(P, e_s)))
lhs = add(add(multiply(A, s_m_s), multiply(G1, s_b_s)), neg(C1_s))          # == e_s * P
prod = pairing(Y, lhs) * pairing(X, multiply(A, e_s)) * pairing(G2, neg(multiply(B, e_s)))
assert prod == FQ12.one(), "R-HVZK: simulated transcript must verify"
print("Theorem R-HVZK (simulator):          PASS (accepting transcript from (A, B, P) alone)")

# --- 7. Theorem R-unforge's simulation: a transcript for E(m) built from a
#         credential on an UNRELATED m_d passes every algebraic check for its
#         chosen challenge; only the hash check tells it apart, and that is
#         what the reduction programs in the random-oracle model ---
m_d = rand()                            # the dummy message
sigma_d = (h, multiply(h, (x + m_d * y) % ORDER))          # a credential on m_d, not m
a_s, b_s, sk_s, r_s = rand(), rand(), rand(), rand()
A_s, B_s = present(sigma_d, a_s, b_s)          # (A, B) from sigma_d
P_s = add(multiply(A_s, m_d), multiply(G1, b_s))          # P := m_d*A + b*G
pk_s = multiply(G1, sk_s)
R_s, C_s = multiply(G1, r_s), add(multiply(G1, m), multiply(pk_s, r_s))          # E encrypts the TARGET m
e_x, s_m_x, s_b_x, s_r_x, s_sk_x = rand(), rand(), rand(), rand(), rand()
C1_x = add(add(multiply(A_s, s_m_x), multiply(G1, s_b_x)), neg(multiply(P_s, e_x)))
T_C_x = add(add(multiply(G1, s_m_x), multiply(pk_s, s_r_x)), neg(multiply(C_s, e_x)))
T_R_x = add(multiply(G1, s_r_x), neg(multiply(R_s, e_x)))
T_k_x = add(multiply(G1, s_sk_x), neg(multiply(pk_s, e_x)))
ok_b = eq(add(multiply(G1, s_m_x), multiply(pk_s, s_r_x)), add(multiply(C_s, e_x), T_C_x))
ok_c = eq(multiply(G1, s_r_x), add(multiply(R_s, e_x), T_R_x))
ok_k = eq(multiply(G1, s_sk_x), add(T_k_x, multiply(pk_s, e_x)))
lhs_x = add(add(multiply(A_s, s_m_x), multiply(G1, s_b_x)), neg(C1_x))
ok_a = (pairing(Y, lhs_x) * pairing(X, multiply(A_s, e_x))
        * pairing(G2, neg(multiply(B_s, e_x)))) == FQ12.one()
assert ok_b and ok_c and ok_k and ok_a
hash_matches = (int(hashlib.sha256(repr((A_s, B_s, R_s, C_s, pk_s, C1_x, T_C_x, T_R_x, T_k_x)).encode()).hexdigest(), 16) % ORDER

```

```

assert not hash_matches                                # the FS check is the only failing check
print("Theorem R-unforge (simulation):                PASS (transcript from a credential on m_d != m passes (a'),(b),(c),(k); only the ha

# --- 8. Proposition R-fresh: reusing b across two presentations identifies m ---
a1, a2 = rand(), rand()
A1, B1 = present(sigma, a1, b)                        # same b as step 1
A2, B2 = present(sigma, a2, b)
P1 = add(multiply(A1, m), multiply(G1, b))
P2 = add(multiply(A2, m), multiply(G1, b))
D_P, D_A = add(P1, neg(P2)), add(A1, neg(A2))
hits = [cand for cand in (m, m_prime, m_d) if eq(add(D_P, neg(multiply(D_A, cand))), bc.Z1)]
assert hits == [m]
print("Proposition R-fresh (b reused):                PASS (P1 - P2 = m*(A1 - A2) singles out the true m among 3 candidates)")

Theorem 1' (masked completeness):                    PASS
Lemma 2 + R1 (raw candidate test):                   PASS (t*sigma identified; (A, B) not)
Theorem 2' (perfect hiding):                         PASS (same (A, B, P) opens to m and m')
Theorem R-sound (extraction):                       PASS ((A, B - b*Y1) is a PS signature on m*)
Guard (A != 0):                                     PASS (A = 0 verifies for every m; rejected)
Theorem R-HVZK (simulator):                         PASS (accepting transcript from (A, B, P) alone)
Theorem R-unforge (simulation):                     PASS (transcript from a credential on m_d != m passes (a'),(b),(c),(k); only the hash diff
Proposition R-fresh (b reused):                     PASS (P1 - P2 = m*(A1 - A2) singles out the true m among 3 candidates)

```

How to read the output: Eight PASS lines. (1) Theorem 1' on an honest presentation. (2) The R1 witness: the raw PS test identifies a rerandomized pair from m alone and fails on the presentation. (3) Theorem 2' made concrete: a second identity m' with blinding $b' = b + (m - m') \log_G A$ opens the *same* (A, B, P) and passes the masked equation. (4) Two transcripts sharing C_1 extract (m^*, b^*) and $(A, B - b^*Y_1)$ verifies as a plain PS signature (Theorem R-sound). (5) The $A \neq O$ guard: the masked equation holds for every m at $A = O$. (6) The Theorem R-HVZK simulator, from (A, B, P) and a challenge alone. (7) Theorem R-unforge's simulation: from a credential on an unrelated m_d , a transcript whose ciphertext encrypts the target m passes every algebraic check for its chosen challenge; only the hash check tells it apart, which is exactly what the reduction programs in the random-oracle model. (8) Proposition R-fresh: a reused b singles out the true m among candidates.

3 Part II: Chaum-Pedersen Re-Encryption Correctness

3.1 Setup

Alice holds a credential whose ElGamal ciphertext is registered on chain and was verified at registration time against her PS signature:

$$E_{\text{alice}} = (R_a, C_a) = (r_a \cdot G, M + r_a \cdot pk_{\text{alice}}), \quad pk_{\text{alice}} = sk_{\text{alice}} \cdot G.$$

To identify herself to Bob she re-encrypts under pk_{bob} with fresh randomness r_b :

$$E_{\text{bob}} = (R_b, C_b) = (r_b \cdot G, M + r_b \cdot pk_{\text{bob}}).$$

She then produces a non-interactive Chaum-Pedersen proof that E_{alice} and E_{bob} encrypt the *same* message point M – without revealing M , m , sk_{alice} , or r_b . The BUCK contract reads E_{alice} from storage (not calldata) and verifies the proof.

3.2 The Statement

Alice proves knowledge of (sk_{alice}, r_b) satisfying the three simultaneous relations

$$pk_{\text{alice}} = sk_{\text{alice}} \cdot G \tag{S1}$$

$$R_b = r_b \cdot G \tag{S2}$$

$$C_a - sk_{\text{alice}} \cdot R_a = C_b - r_b \cdot pk_{\text{bob}}. \tag{S3}$$

(S1) binds her to the registered public key; (S2) binds r_b to the ciphertext she submits; (S3) states that the ElGamal-decrypt of E_{alice} equals the ElGamal-decrypt of E_{bob} under their respective secret keys.

3.3 Protocol (Fiat-Shamir)

Commit. Prover samples $k_1, k_2 \leftarrow \mathbb{Z}_q$ and computes

$$T_1 = k_1 \cdot G, \quad T_2 = k_2 \cdot G, \quad T_3 = k_2 \cdot pk_{\text{bob}} - k_1 \cdot R_a.$$

Challenge.

$$e = H(R_a, C_a, R_b, C_b, pk_{\text{alice}}, pk_{\text{bob}}, T_1, T_2, T_3, \text{msg.sender}, \text{spender}, \text{chainid}, \text{registry}, \text{keccak}(\tau_{\text{approve}})),$$

with $\tau_{\text{approve}} = \text{AlbertaBuck/FiatShamir/IdentityRegistry/Approve/v2}$.

Respond. $s_1 = k_1 - e \cdot sk_{\text{alice}} \pmod{q}$, $s_2 = k_2 - e \cdot r_b \pmod{q}$.

Verify. The contract recomputes e from the public inputs and the prover-supplied commitments, then checks

$$s_1 \cdot G + e \cdot pk_{\text{alice}} \stackrel{?}{=} T_1 \tag{V1}$$

$$s_2 \cdot G + e \cdot R_b \stackrel{?}{=} T_2 \tag{V2}$$

$$s_2 \cdot pk_{\text{bob}} - s_1 \cdot R_a + e \cdot (C_b - C_a) \stackrel{?}{=} T_3. \tag{V3}$$

3.4 Theorem 3 (Completeness)

Statement. An honestly generated transcript satisfies (V1), (V2), (V3).

Proof. (V1) and (V2) are standard Schnorr identities:

$$\begin{aligned} s_1 G + e \cdot pk_{\text{alice}} &= (k_1 - e \cdot sk_{\text{alice}})G + e \cdot sk_{\text{alice}}G = k_1 G = T_1, \\ s_2 G + e \cdot R_b &= (k_2 - er_b)G + er_b G = k_2 G = T_2. \end{aligned}$$

For (V3), substitute s_1, s_2 and rearrange:

$$\begin{aligned} s_2 \cdot pk_{\text{bob}} - s_1 \cdot R_a + e(C_b - C_a) &= (k_2 - er_b)pk_{\text{bob}} - (k_1 - e \cdot sk_{\text{alice}})R_a + e(C_b - C_a) \\ &= [k_2 pk_{\text{bob}} - k_1 R_a] + e[sk_{\text{alice}}R_a - r_b \cdot pk_{\text{bob}} + C_b - C_a] \\ &= T_3 + e[-(C_a - sk_{\text{alice}}R_a) + (C_b - r_b \cdot pk_{\text{bob}})] \\ &= T_3 + e \cdot [-M + M] \\ &= T_3. \end{aligned}$$

where the penultimate line uses the witness relations from the honest setup, and the final line uses (S3). ■

3.5 Theorem 4 (Special Soundness / 2-Extractability)

Statement. Given two accepting transcripts $(T_1, T_2, T_3, e, s_1, s_2)$ and $(T_1, T_2, T_3, e', s'_1, s'_2)$ with $e \neq e'$ (and identical public inputs), there is an efficient algorithm that outputs a witness (sk^*, r^*) satisfying (S1), (S2), (S3).

Proof (extractor). Subtract (V1) for the two transcripts:

$$(s_1 - s'_1)G + (e - e') \cdot pk_{\text{alice}} = O \implies pk_{\text{alice}} = \frac{s'_1 - s_1}{e - e'} \cdot G.$$

Set $sk^* := (s'_1 - s_1)(e - e')^{-1} \bmod q$; this is well-defined because q is prime and $e \neq e'$. By construction (S1) holds. Similarly from (V2), $r^* := (s'_2 - s_2)(e - e')^{-1} \bmod q$ satisfies (S2): $R_b = r^* G$.

For (S3), subtract (V3) for the two transcripts:

$$(s_2 - s'_2) \cdot pk_{\text{bob}} - (s_1 - s'_1) \cdot R_a + (e - e')(C_b - C_a) = O.$$

Substitute $s_2 - s'_2 = -(e - e')r^*$ and $s_1 - s'_1 = -(e - e')sk^*$:

$$-(e - e')r^* \cdot pk_{\text{bob}} + (e - e')sk^* \cdot R_a + (e - e')(C_b - C_a) = O.$$

Dividing through by the nonzero scalar $(e - e')$:

$$sk^* \cdot R_a - r^* \cdot pk_{\text{bob}} + (C_b - C_a) = O \iff C_a - sk^* \cdot R_a = C_b - r^* \cdot pk_{\text{bob}},$$

which is (S3). Therefore (sk^*, r^*) is a valid witness, and the extractor runs in the time of four field inversions and a constant number of group operations. ■

Remark (implication for cheating provers). Special soundness lifts, via the forking lemma⁵, to a reduction from any efficient prover that convinces the verifier on a false statement to an efficient solver for discrete logarithms on G_1 . Under the DLog assumption on alt_bn128 , no such prover exists. In particular, Alice cannot produce a valid proof for E_{bob} encrypting any $M' \neq M$ – not merely "hard to compute," but infeasible under a standard assumption Ethereum already requires.

3.6 Theorem 5 (Honest-Verifier Zero-Knowledge)

Statement. There is an efficient simulator $\mathcal{S}$ that, given only the public inputs and a challenge e drawn from the verifier’s distribution, outputs a transcript $(T_1, T_2, T_3, e, s_1, s_2)$ distributed *identically* to an honest prover’s transcript conditioned on that challenge.

Proof (simulator). $\mathcal{S}$ samples $s_1, s_2 \leftarrow \mathbb{Z}_q$ uniformly, then *defines*

$$\begin{aligned} T_1 &:= s_1 \cdot G + e \cdot pk_{\text{alice}}, \\ T_2 &:= s_2 \cdot G + e \cdot R_b, \\ T_3 &:= s_2 \cdot pk_{\text{bob}} - s_1 \cdot R_a + e \cdot (C_b - C_a). \end{aligned}$$

Equations (V1), (V2), (V3) hold by construction, so the transcript is accepting.

Distribution match. In the real protocol, k_1, k_2 are uniform on $\mathbb{Z}_q$; the honest responses $s_i = k_i - e \cdot w_i$ are therefore uniform on $\mathbb{Z}_q$ (shift by a constant). The commitments T_1, T_2, T_3 are *deterministic functions* of the s_i and the public inputs: (V1), (V2), (V3) each uniquely determine one T_i from the corresponding equation. The simulator samples s_i uniformly and computes the T_i via the same equations – the joint distribution over $(T_1, T_2, T_3, s_1, s_2)$ is identical. ■

Corollary (NIZK via Fiat-Shamir in the ROM). Model H as a programmable random oracle. The Fiat-Shamir transform replaces the verifier’s challenge with $e = H(\text{public inputs}, T_1, T_2, T_3, \text{context})$. Standard arguments⁶ promote:

- HVZK $\Rightarrow$ NIZK (the simulator programs H so that $H(\text{public inputs}, T_1, T_2, T_3, \dots) = e$ for the (T_i) it produced);
- special soundness $\Rightarrow$ proof-of-knowledge soundness (the forking lemma rewinds the oracle).

The binding of `msg.sender`, `spender`, `chainid` into the hash input is standard domain separation that prevents cross-context replay.

3.7 Sanity Check (py_ecc on BN254)

```
# Build registered credential E_alice for Alice
sk_alice = rand()
pk_alice = multiply(G1, sk_alice)
r_a      = rand()
R_a      = multiply(G1, r_a)
M        = multiply(G1, m)                # identity point M = m * G
C_a      = add(M, multiply(pk_alice, r_a))

# Bob's identity public key (Alice knows this from the registry)
sk_bob = rand()
pk_bob = multiply(G1, sk_bob)

# Re-encryption for Bob: E_bob = (r_b * G, M + r_b * pk_bob)
r_b = rand()
R_b = multiply(G1, r_b)
C_b = add(M, multiply(pk_bob, r_b))

def cp_prove(k1, k2):
    T1 = multiply(G1, k1)
    T2 = multiply(G1, k2)
```

⁶Fiat, A. and Shamir, A. "How to Prove Yourself: Practical Solutions to Identification and Signature Problems." CRYPTO 1986. The transform replacing an interactive verifier’s challenge with a hash of the transcript, analyzed in the random oracle model of Bellare and Rogaway (CCS 1993).

```

T3 = add(multiply(pk_bob, k2), neg(multiply(R_a, k1)))
tag = "|".join(str(v) for v in
                [G1, pk_alice, pk_bob, R_a, C_a, R_b, C_b, T1, T2, T3])
e = int(hashlib.sha256(tag.encode()).hexdigest(), 16) % ORDER
s1 = (k1 - e * sk_alice) % ORDER
s2 = (k2 - e * r_b) % ORDER
return (T1, T2, T3, e, s1, s2)

def cp_verify(T1, T2, T3, e, s1, s2):
    v1 = eq(add(multiply(G1, s1), multiply(pk_alice, e)), T1)
    v2 = eq(add(multiply(G1, s2), multiply(R_b, e)), T2)
    Cdiff = add(C_b, neg(C_a))
    v3 = eq(add(add(multiply(pk_bob, s2), neg(multiply(R_a, s1))),
                multiply(Cdiff, e)), T3)
    return v1, v2, v3

# --- Theorem 3: completeness ---
proof = cp_prove(rand(), rand())
v1, v2, v3 = cp_verify(*proof)
assert v1 and v2 and v3, "Theorem 3 FAILED"
print("Theorem 3 (completeness):          PASS")

# --- Theorem 4: extract witness from two transcripts with same (T1,T2,T3) ---
# We fix (k1, k2) to produce the same (T1, T2, T3), then force two
# different challenges by hashing with two different context strings
# (in production the different challenges arise from rewinding the R0).
k1, k2 = rand(), rand()
T1 = multiply(G1, k1)
T2 = multiply(G1, k2)
T3 = add(multiply(pk_bob, k2), neg(multiply(R_a, k1)))

def responses(e):
    return (k1 - e * sk_alice) % ORDER, (k2 - e * r_b) % ORDER

e_1 = rand()
e_2 = rand()
while e_1 == e_2:
    e_2 = rand()
s1_a, s2_a = responses(e_1)
s1_b, s2_b = responses(e_2)

def inv_mod(a, p):
    return pow(a, -1, p)

delta_e = (e_1 - e_2) % ORDER
delta_e_inv = inv_mod(delta_e, ORDER)
sk_star = ((s1_b - s1_a) * delta_e_inv) % ORDER # should equal sk_alice
r_star = ((s2_b - s2_a) * delta_e_inv) % ORDER # should equal r_b

assert sk_star == sk_alice, "Theorem 4 sk extraction FAILED"
assert r_star == r_b, "Theorem 4 r extraction FAILED"
# Witnesses satisfy (S1), (S2), (S3):
assert eq(multiply(G1, sk_star), pk_alice)
assert eq(multiply(G1, r_star), R_b)
assert eq(add(C_a, neg(multiply(R_a, sk_star))),
          add(C_b, neg(multiply(pk_bob, r_star))))
print("Theorem 4 (special soundness):      PASS (sk, r extracted from two transcripts)")

# --- Theorem 5: simulator produces accepting transcript without witness ---
def cp_simulate(e):
    s1 = rand(); s2 = rand()
    T1 = add(multiply(G1, s1), multiply(pk_alice, e))
    T2 = add(multiply(G1, s2), multiply(R_b, e))
    Cdiff = add(C_b, neg(C_a))
    T3 = add(add(multiply(pk_bob, s2), neg(multiply(R_a, s1))),
            multiply(Cdiff, e))
    return (T1, T2, T3, e, s1, s2)

```

```

sim = cp_simulate(rand())
v1, v2, v3 = cp_verify(*sim)
assert v1 and v2 and v3, "Theorem 5 FAILED (simulated transcript rejected)"
print("Theorem 5 (HVZK):", "PASS (simulator accepts without witness)")

# --- Unsoundness counter-example: wrong M at destination ---
M_fake = multiply(G1, rand())
C_b_fake = add(M_fake, multiply(pk_bob, r_b))
# Honest prover tries to produce a proof for (E_alice, (R_b, C_b_fake))
k1, k2 = rand(), rand()
T1 = multiply(G1, k1)
T2 = multiply(G1, k2)
T3 = add(multiply(pk_bob, k2), neg(multiply(R_a, k1)))
tag = "|".join(str(v) for v in
               [G1, pk_alice, pk_bob, R_a, C_a, R_b, C_b_fake, T1, T2, T3])
e_f = int(hashlib.sha256(tag.encode()).hexdigest(), 16) % ORDER
s1_f = (k1 - e_f * sk_alice) % ORDER
s2_f = (k2 - e_f * r_b) % ORDER
v1_f = eq(add(multiply(G1, s1_f), multiply(pk_alice, e_f)), T1)
v2_f = eq(add(multiply(G1, s2_f), multiply(R_b, e_f)), T2)
Cdiff_f = add(C_b_fake, neg(C_a))
v3_f = eq(add(add(multiply(pk_bob, s2_f), neg(multiply(R_a, s1_f))),
               multiply(Cdiff_f, e_f)), T3)
assert v1_f and v2_f and not v3_f, "Unsoundness: fake M proof should fail at V3"
print("Counter-example (M != M'):", "V1,V2 PASS; V3 FAIL (as required)")

Theorem 3 (completeness): PASS
Theorem 4 (special soundness): PASS (sk, r extracted from two transcripts)
Theorem 5 (HVZK): PASS (simulator accepts without witness)
Counter-example (M != M'): V1,V2 PASS; V3 FAIL (as required)

```

How to read the output: Four PASS lines plus the counter-example. The counter-example is the mirror image of Theorem 4: when the statement is *false* (different message points), the first two verification equations still pass (Alice's key and randomness are genuine) but (V3) – the "same-message" equation – fails. This is precisely the mathematical impossibility the prose in the Identity document refers to when it says "no valid proof exists."

4 Part III: End-to-End Re-Encryption Correctness

4.1 Chain of Trust

The on-chain identity pipeline chains together the two primitives proved above:

KYC ceremony:	issuer signs m	-->	σ on m	(PS-CM unforgeability)
Fountain:	present σ	-->	(A, B) hiding m	(Theorems 1', 2')
Registration:	masked-PS NIZK	-->	E_{alice} bound to same m	(Theorems R-sound, R-comp)
approve(Bob):	Alice re-encrypts	-->	E_{bob}	(Theorem 3)
	Chaum-Pedersen	-->	verified on chain	(Theorems 3, 4, 5)
Decryption:	Bob computes M	-->	$M = C_b - sk_{\text{bob}} \cdot R_b$	(ElGamal correctness)
Two-channel:	Bob receives	-->	$m(\text{identity_data}) \cdot G == M$	(tagged hash commitment)

4.2 Theorem 6 (End-to-End Re-Encryption Correctness)

Statement. Suppose the BUCK contract:

1. holds a registered credential $E_{\text{alice}} = (R_a, C_a)$ together with pk_{alice} , accepted at registration time with a presentation (A, B) and a registration NIZK that verified against a trusted issuer public key (X, Y, Y_1) ;
2. accepts a Chaum-Pedersen proof π supplied by `msg.sender` = Alice, with spender pk_{bob} and submitted ciphertext $E_{\text{bob}} = (R_b, C_b)$.

Then, under the DLog assumption on G_1 and in the ROM, there exists $M = m \cdot G$ such that

$$C_a - sk_{\text{alice}} \cdot R_a = M = C_b - sk_{\text{bob}} \cdot R_b,$$

and the scalar m is an identity scalar the issuer certified.

Proof sketch. By Theorems R-sound and R-comp, an accepting registration yields (through the forking lemma) a single witness (m, b, r, sk) with $C_a - sk_{\text{alice}} R_a = m \cdot G$ and $(A, B - bY_1)$ a plain PS signature on m under (X, Y) ; PS-CM unforgeability ensures m is a scalar the issuer signed. The verifier never sees m : it checks the masked equation (a'), not a signature against a visible message. The contract reads E_{alice} from storage, not calldata, so the value is frozen.

Theorem 4 (special soundness) plus the forking lemma reduction to DLog gives: from any adversary producing an accepting π with non-negligible probability there is an extractor outputting witnesses (sk^*, r^*) with $C_a - sk^* R_a = C_b - r^* pk_{\text{bob}}$. Since $pk_{\text{alice}} = sk^* \cdot G = sk_{\text{alice}} \cdot G$ and G is a generator, $sk^* = sk_{\text{alice}}$; so both sides equal the registered M . Therefore E_{bob} decrypts under sk_{bob} to the same M that the issuer's PS signature attests to. ■

4.3 Why This Suffices for Every Downstream Use Case

- **approve / transfer / transferFrom (Identity):** Bob decrypts E_{bob} and obtains M ; combined with off-chain `identity_data` delivery and the check $\text{keccak}(\tau_{\text{ID}} \parallel \text{identity_data}) \cdot G = M$, he learns Alice's real identity with cryptographic assurance. A BUCK **transfer / transferFrom** is gated on this **approve** handshake against *both* parties' registered records, so a value move carries an account $\leftrightarrow$ Identity binding for *each* counterparty – the bilateral mutual decryptability formalised in the EOA mirror below.

- **Notes spend predicates (Part IV):** The same re-encryption / decryption algebra underlies BUCK Notes. At mint the issuer addresses the recipient's receiving key and a mint binding forces the ciphertext over the issuer's *own* registered M_{iss} (Theorem 7). At spend, relation (3) of the folded gate is this Part's statement – the depositing account's registered credential decrypts to the Identity m_{rec} – and the gate ties that Identity to the reading key through a certified accumulator leaf (Theorem 8); the bearer B1 path instead binds the depositor's *own* Identity M_{dep} for the public issuer (Theorem 9). The binding is to the *Identity*, not a mint-time keypair – *recovery, not finality* on key loss – yet identity laundering through the note pool remains infeasible.
- **Identity Guardians (approveFor, recovery, cosigning):** The Schnorr proof of knowledge of sk is a one-statement specialization of the above (only (S1) with a Fiat-Shamir binding to $(\text{msg.sender}, \text{principal})$). Theorems 3-5 apply verbatim with the second and third statements dropped.

4.4 The EOA Transfer Mirror (transfer / transferFrom)

A private $\rightarrow$ private BUCK **transfer** achieves the *same* mutual-decryptability invariant as a Notes spend, but *without* a SNARK and with collusion-resistance for free – precisely because the transfer is *not* anonymous: both counterparties are named at one moment. The bilateral **approve** handshake reads *both* registered records in a single call and runs the re-encryption of this Part in each direction; the value move is gated on it.

Step	Cryptographic step	Who is named	Guarantee
register	store $(pk, E_{\text{addr}} = \text{Enc}(M, pk)) + \text{PS credential}$	–	Identity $\leftrightarrow$ KYC (Theorem 6)
approve (bilateral)	re-encrypt $E_{A \rightarrow B} = \text{Enc}(M_A, pk_B) + \text{Chaum-Pedersen} \equiv E_{\text{addr}}[A]$	both	sound re-encryption under pk_B
transfer / transferFrom	value $A \rightarrow B$, gated on the approve	both (linked)	mutual decryptability established
receipt	verifiable_decrypt $E_{X \rightarrow \text{me}} \rightarrow M$	–	either party names the other

Corollary (EOA transfer mutual decryptability). For a **transfer / transferFrom** gated on bilateral **approve** proofs, each counterparty can recover the other's registered Identity point (B decrypts $E_{A \rightarrow B}$ under sk_B to M_A ; A decrypts $E_{B \rightarrow A}$ under sk_A to M_B), each can prove it under compulsion (verifiable decryption), and no third party lacking either secret key recovers M_A or M_B (ElGamal IND-CPA, hence DDH). This is the BUCK mutual-decryptability invariant in its *cheapest* form; the Identity-M Notes (Part IV) reconstruct the *same* invariant across an anonymising mint $\rightarrow$ spend detour, which is exactly what forces the extra apparatus there (Theorems 7-11).

Proof. Each direction is Theorem 6 applied to the respective ordered pair: an accepted Chaum-Pedersen proof certifies that $E_{A \rightarrow B}$ re-encrypts A 's registered, PS-certified M_A under pk_B , so $C_{A \rightarrow B} - sk_B R_{A \rightarrow B} = M_A$ by ElGamal correctness; symmetrically for $B \rightarrow A$. Third-party secrecy is the IND-CPA security of ElGamal on G_1 : deciding whether a ciphertext encrypts a candidate M is a DDH decision (see *Harvest-Now, Decrypt-Later* for the quantum caveat). ■

4.5 Sanity Check (py_ecc on BN254)

```
# Reuse M (Alice's identity M_A), (sk_alice, pk_alice), (sk_bob, pk_bob),
# and Alice's re-encryption to Bob E_bob = (R_b, C_b) from Part II.
def decrypt(R, C, sk): return add(C, neg(multiply(R, sk))) # ElGamal decrypt
def reencrypt(P, pk):
    r = rand(); return (multiply(G1, r), add(P, multiply(pk, r)))

M_A = M # Alice's Identity point
m_bob = int(hashlib.sha256(b"Bob Smith | Alberta | AIC-2026").hexdigest(), 16) % ORDER
M_B = multiply(G1, m_bob) # Bob's Identity point
```

```

# approve(Alice -> Bob): Bob decrypts Alice's re-encryption to M_A
assert eq(decrypt(R_b, C_b, sk_bob), M_A)
# approve(Bob -> Alice): Alice decrypts Bob's re-encryption to M_B
R_ba, C_ba = reencrypt(M_B, pk_alice)
assert eq(decrypt(R_ba, C_ba, sk_alice), M_B)
print("Corollary (transfer mutual decryptability):          PASS")

# Mallory holds neither sk: a wrong key recovers a wrong point (DDH)
assert not eq(decrypt(R_b, C_b, sk_alice), M_A)
assert not eq(decrypt(R_ba, C_ba, sk_bob), M_B)
print("Corollary (third-party cannot decrypt):             PASS")

Corollary (transfer mutual decryptability):          PASS
Corollary (third-party cannot decrypt):             PASS

```

DRAFT

5 Part IV: BUCK Notes Spend Predicates (A1, A2, B1)

This Part proves what an accepted Notes spend establishes about the Identities on either side of it. An addressed note (A1, A2) is encrypted to the recipient’s *receiving key*, which is independent of its Identity, and its deposit is ONE folded Groth16 proof over one witness. A bearer note (B1) is spent with a sigma protocol and a membership proof that share one committed point. The clear-text reference for the folds is `alberta_buck/wallet/deposit_fold.py`; the architecture, and the attack each relation refuses, are in the receiving-key architecture.

5.1 Setup

Pool. noteTree, a depth-20 Poseidon tree of leaves $cm = \text{Poseidon}_6(T_{CM}, \text{flavor}, v, \rho, \text{idHash}, \text{predicate})$ (idHash digests the per-flavor identity payload), whose empty leaf is the field tag of `AlbertaBuck/Notes/Zero/v2`; a `nullifiers` set; a 30-deep ring of recent roots. Mint is the batch rollup of *alberta-buck-notes-rollup-mint* – a Groth16 proof of incremental insertion of N fresh leaves with in-circuit range checks ($v_i, \text{totalFace} \in [0, 2^{128})$) and value conservation ($\sum_i v_i = \text{totalFace}$); the contract pins `oldRoot` / `nextLeafIndex` and pulls `totalFace` BUCK. Demurrage is orthogonal.

Identity accumulator. Membership is a path through two trees. Each authority keeps subtrees: an identity registry’s are private, of depth 12, with salted leaves; an institution’s are public, with unsalted ones. The aggregator’s tree, of depth 20, has subtree roots for leaves, and its root `rtid` is posted on chain into a ring of 256 records, each with its posting time. A consumer declares the oldest root it accepts – Notes, seven days – and a spend names the root it proved against. Every level folds with the same Poseidon, so a membership proof is one path of 32 levels. Each leaf function leads with its own `v2` field tag τ :

$$\ell_R(m, k, s) = \text{Poseidon}(\tau_R, m, k, s), \quad \ell_S(M, s) = \text{Poseidon}(\tau_S, M.x, M.y, s), \quad \ell(M) = \text{Poseidon}(\tau_I, M.x, M.y).$$

ℓ_R , the receiving leaf, pairs an Identity with a receiving key; ℓ_S is the salted identity leaf; ℓ the public one. A salt derives from a wallet secret, never from m , so a party holding every certified identity scalar and a whole published subtree decides no membership (Poseidon as a random oracle). **register** refuses a caller-supplied leaf: admission is the authority’s certification, not a by-product of the hiding presentation, whose leaf would otherwise reveal M to whoever computed it.

Identity primitives (recap, Parts I-III). A registered account is (`addr`, $pk = sk G$, $E_{\text{addr}} = (rG, M + rpk)$): an ElGamal encryption of its issuer-certified Identity point $M = mG$ under its own key. The Identity Fountain issues unlimited fresh accounts ($sk_i, pk_i, E_{\text{addr},i}$) on the *same* M .

Receiving key. The recipient holds a receiving secret k , derived from wallet seed material and independent of m , with $pk_{\text{recv}} = kG$; its registry certifies $\ell_R(m_{\text{rec}}, k, s)$. The identity scalar is disclosed to every counterparty by design (Part II), so nothing is encrypted under M : a key everyone you pay learns cannot also read your mail.

Note addressing (mint).

$$\text{A1: } eNote = (r_n G, vG + r_n pk_{\text{recv}}), \quad eRec = (r' G, M_{\text{rec}} + r' pk_{\text{recv}});$$

$$\text{A2: } eNote = (r_n G, vG + r_n pk_{\text{recv}}), \quad eIss = (R_i, C_i) = (r' G, M_{\text{iss}} + r' pk_{\text{recv}}).$$

A1’s issuer is public: it signs the batch with a Schnorr proof checked at mint, and `idHash` = $\text{Poseidon}_6(T_{ID}, eNote, m_{\text{iss}})$. A2’s issuer is private: a recipient-blinded re-encryption binding (`verifyIssuerReenc`) publishes $Q = pk_{\text{recv}} + \beta H$, $U = r' H$, $T = r' pk_{\text{recv}} + \gamma H$, and `idHash` = $\text{Poseidon}_{11}(T_{ID}, eNote, eIss, T)$; γ reaches the recipient wrapped in the delivery. B1 is bearer: no recipient at mint, the public issuer

signs the batch, and the depositor encrypts its *own* Identity for the issuer at spend (Theorem 9). Throughout this Part $H = H_{\text{PED}}$.

Nullifier. Flavor-agnostic: $nf = \text{Poseidon}_3(T_{NF}, \rho, \text{idHash})$. Each note hash leads with its own v2 domain word (T_{CM}, T_{ID}, T_{NF}), so no two of them, and none of them and any other Poseidon in the protocol, share a preimage.

Transcripts. Every sigma protocol's Fiat-Shamir challenge ends in its own v2 domain word, so no accepting transcript of one protocol is an accepting transcript of another.

5.2 Theorem 7 (Mint Addressing and Anti-Framing)

Statement. Let the recipient hold (m_{rec}, k) and the issuer a registered credential $E_{\text{iss}}^{\text{reg}} = (R_{\text{reg}}, C_{\text{reg}})$ decrypting under sk_{iss} to its certified M_{iss} . For fresh r_n, r' :

1. **(Decryptability.)** k decrypts $eNote$ to vG , $eRec$ to M_{rec} , and $eIss$ to M_{iss} – from whichever account the recipient uses.
2. **(Anti-framing.)** An accepted mint binding forces the encrypted point to be the issuer's *own* registered Identity: `verifyIssuerSchnorr` for a public issuer; for A2, `verifyIssuerReenc` forces $eIss = (r'G, M_{\text{iss}} + r'pk_Q)$ with M_{iss} the plaintext of $E_{\text{iss}}^{\text{reg}}$ at `msg.sender` and $pk_Q = Q - \beta H$.
3. **(Issuer ignorance, unlinkability.)** The issuer learns pk_{recv} , and M_{rec} off chain, but neither k nor m_{rec} . Q, U, T are uniform, and two mints to one recipient are indistinguishable under DDH.

Proof. (1) ElGamal correctness under $pk_{\text{recv}} = kG$: $C - kR = P + r'kG - kr'G = P$. (2) Schnorr EUF-CMA for the public binding. For A2, special soundness of the five-relation Okamoto sigma extracts $(r', \beta, sk_{\text{iss}}, \gamma)$ with

$$\text{L1: } R_i = r'G, \quad \text{L2: } U = r'H, \quad \text{L3: } T = r'Q - \beta U + \gamma H, \quad \text{L4: } pk_{\text{iss}} = sk_{\text{iss}}G, \quad \text{L5: } C_{\text{reg}} + T - C_i = sk_{\text{iss}}R_{\text{reg}} + \gamma H.$$

L2 and L3 give $T = r'pk_Q + \gamma H$; L5 then gives $C_i = (C_{\text{reg}} - sk_{\text{iss}}R_{\text{reg}}) + r'pk_Q = M_{\text{iss}} + r'pk_Q$. (3) The issuer handles group elements only; β and γ are one-time pads on Q and T , U is uniform for uniform r' , and distinguishing two mints is IND-CPA. ■

Remark (which key). (2) binds $eIss$ to pk_Q , a key the minter chose, and not necessarily to pk_{recv} . An ElGamal ciphertext does not commit to its key, so the same bytes can open under k to a different registered Identity. Theorem 12 closes this at spend.

5.3 Theorem 8 (Folded Deposit Gate – A1, A2)

Spending an addressed note needs two facts about two DIFFERENT secrets: "I can read this note" (k) and "I am this registered Identity" (m_{rec}). Proved side by side they say nothing about their owner – a thief holding a stolen payload answers the first with the stolen k and the second with its own Identity, and both are true. So the gate is one proof over one witness, and the tie is a relation.

Statement. Suppose `Notes` accepts an addressed spend (`spendCoupledA1` / `spendCoupledA2`): the spend proof (cm under `noteRoot`, nf well formed, flavor bound to the entry point); `rtid` accepted for the `Notes` consumer; and the fold proof, whose public inputs the adapter derives on chain – nf , (A1) the face v , `rtid`, the supplied $eEnc = (R_e, C_e)$, and the depositor's pk_{dep} and $E_{\text{dep}} = (R_d, C_d)$, read from the registry at `msg.sender`. Under SNARK knowledge soundness and Poseidon collision resistance there is an extractor producing a witness with:

1. **(Same note.)** $nf = \text{Poseidon}_3(T_{NF}, \rho, \text{idHash})$ for the (ρ, idHash) the spend proof opened, and idHash opens to the flavor's payload: A1 $(eNote, m_{\text{iss}})$; A2 $(eNote, eIss, T)$.
2. **(Reading.)** A1: $eNote = (r_n G, (v + r_n k)G)$. A2: $eIss = (r'G, M_I + (r'k)G)$ for a point M_I .
3. **(The spend's ciphertext.)** A1: $eEnc = (tG, (m_{\text{rec}} + tk)G)$. A2: $eEnc = (tG, M_I + (tk)G)$.
4. **(Account.)** $pk_{\text{dep}} = sk_{\text{dep}}G$, $R_d = r_E G$, $C_d = (m_{\text{rec}} + sk_{\text{dep}}r_E)G$: the depositing account's registered credential decrypts to $M_{\text{rec}} = m_{\text{rec}}G$.
5. **(Tie.)** $\ell_R(m_{\text{rec}}, k, s)$ has a 32-level path to rt_{id} .
6. **(A2: issuer registered.)** $\ell_S(M_I, s_{\text{iss}})$ has a 32-level path to the same rt_{id} .
7. **(A2: key tie.)** $T = (r'k)G + \gamma H$.

Proof. Knowledge soundness extracts the witness, and each clause is a constraint. The fold and the spend proof share only nf , and Poseidon collision resistance makes the fold's (ρ, idHash) the spend's. The clauses are joined by signal identity rather than by inference across proofs: one m_{rec} signal enters (3) and the leaf of (4), and one k enters (1), (2), (4) and (6). Every witnessed limb is range-checked. A1 folds its point sums into scalar sums and adds no points; A2's three additions each assert distinct x -coordinates, the precondition of the incomplete addition formula. ■

Implications.

- **Only the addressed Identity spends.** A thief holding a stolen payload has ρ , the openings and k , and satisfies (3) only with its own account, whose Identity is $m' \neq m_{\text{rec}}$. Clause (4) then needs $\ell_R(m', k, \cdot)$ certified, and no authority certified that pair. Nor can it bring its own certified pair (m', k') , because the committed payload pins k : in A1, $eNote$ and the public v fix both r_n and $r_n k$; in A2, T fixes $(r'k)G$ up to γH , and moving to k' needs $r'(k - k')G = \delta H$ for a known δ , a discrete logarithm.
- **Recovery, not finality.** Any registered account of M_{rec} deposits (clause 3), and k is recovered from the wallet seed. Losing an account key loses no note.
- **Collusion closed (A2).** An issuer that keys $eIss$ to a throwaway point leaves the recipient decrypting an M_I with no certified leaf; (5) has no witness and the deposit reverts. No spendable-but-unnameable A2 note exists.
- **The issuer cannot self-redeem.** It knows pk_{recv} and M_{rec} , not k or m_{rec} , and (4) needs both – unless it *is* the recipient.
- **Revocation latency is the consumer's age.** A holder cleared from its subtree proves against no root posted after the clearance, and Notes accepts a root for seven days; that age is the protocol's statement of how long a revoked holder can still spend.
- **No third-party leakage.** No public input names an Identity: $eEnc$ is a fresh encryption under pk_{recv} , which is never public; the account's key and credential were public already; no leaf, subtree slot or path is revealed. $eEnc$ is a re-encryption of the note's ciphertext, not the ciphertext itself, so the spend presents nothing the mint published.

5.4 Theorem 9 (Depositor-Binding Soundness – B1)

B1 needs no fold. Its two facts – "my payout account is bound to this Identity" and "this ciphertext encrypts that Identity for the issuer" – rest on ONE secret, m_{dep} , so the sigma's shared Fiat-Shamir nonce is a genuine tie.

Statement. Suppose Notes accepts a bearer spend (`spendCoupledB1`): the spend proof, `verifyDepositorBinding(depositor, issuer, eDepForIss, pi)`, `rtid` accepted, and the `identity_membership` proof over $pi.P_{\text{dep}}$. Under SNARK knowledge soundness, special soundness of the Okamoto sigma, Schnorr EUF-CMA⁷, and DLog with H hashed to the curve, extractors produce $(m_{\text{dep}}, sk_{\text{dep}}, r, b)$ from the sigma and (M, b', s) with a path from the SNARK such that:

1. $cm = \text{Poseidon}_6(T_{CM}, B1, v, \rho, \text{idHash}, \cdot)$ lies under `noteRoot`, and the prover knows ρ (the bearer authorization).
2. (**Account.**) $pk_{\text{dep}} = sk_{\text{dep}}G$, $C_d = m_{\text{dep}}G + sk_{\text{dep}}R_d$.
3. (**Encrypted for the issuer.**) $eDepForIss = (rG, m_{\text{dep}}G + rpk_{\text{iss}})$, the same m_{dep} as (2).
4. (**Committed.**) $P_{\text{dep}} = m_{\text{dep}}G + bH = M + b'H$.
5. (**Registered.**) $\ell_S(M, s)$ has a 32-level path to `rtid`, and $M = M_{\text{dep}}$.
6. The public issuer is named by m_{iss} and an EUF-CMA Schnorr over the batch.

Proof. The sigma's single response for m_{dep} couples (2), (3) and the first opening of (4) to one Identity (Theorem 4 shape). The circuit computes $b'H$ from b' rather than accepting it, and the contract feeds the same P_{dep} to both verifiers. Equating the two openings gives $M - M_{\text{dep}} = (b - b')H$. A reduction that issued or extracted (Theorem R-sound) every registered scalar knows $\log_G M$, so $M \neq M_{\text{dep}}$ yields $\log_G H$, which no one knows: H is hashed to the curve. Schnorr EUF-CMA gives (6). ■

Implications.

- **The public issuer names the bearer depositor.** From the `SpentCoupledB1` event the issuer decrypts $eDepForIss$ under sk_{iss} to M_{dep} – collusion-resistantly, since a substituted ciphertext breaks (3), and privately from third parties (IND-CPA).
- **Collusion closed.** A bogus M_{dep} has no certified leaf: (5) fails and the deposit reverts. No one cashes a B1 note un-nameably.
- **Why the generator must have no known logarithm.** With $H = hG$ for a public h , a depositor holding any registered scalar m' – which counterparties hold by design – sets $b' = b + (m_{\text{dep}} - m')/h$, and the membership proof speaks of m' while the sigma speaks of its own unregistered m_{dep} . Hashing H to the curve leaves no h to shift by.
- **Bearer race is civil-trust.** Whoever publishes ρ first wins; the issuer who chose ρ can race but is identified at the moment of theft. The cryptography places no prior restraint – the legal system does, exactly as for a paper cashier's cheque.

⁷Schnorr, C.-P. "Efficient Signature Generation by Smart Cards." Journal of Cryptology 4(3), 1991, pp. 161-174. The Schnorr signature scheme is EUF-CMA secure under the discrete logarithm assumption in the random oracle model; see Pointcheval and Stern (2000) ⁵ for the forking-lemma reduction, and Bellare and Neven, "Multi-signatures in the Plain Public-Key Model" (CCS 2006) for tightened security bounds.

5.5 Theorem 10 (Nullifier and Double-Spend Prevention)

The nullifier is flavor-agnostic, $nf = \text{Poseidon}_3(T_{NF}, \rho, \text{idHash})$, over the spend proof's opening (idHash digests the identity payload), so one **nullifiers** set serves every flavor.

1. **(Determinism / double-spend.)** nf is a deterministic function of (ρ, idHash) ; two accepting spends of the same cm yield the same nf , and the contract's $nf \notin \text{nullifiers}$ check rejects the second.
2. **(Third-party unlinkability.)** Modelling Poseidon as a random oracle, nf is a PRF over the secret opening; without $(\rho, \text{idHash}) - \rho$ a uniform q -element – inverting nf to its cm is pre-image resistance. So *no party lacking the opening* links a spend to its mint. The two counterparties who hold the opening already know the note exists; the nullifier protects the transaction *graph* from outside observers – exactly what bulk-trace resistance requires.

Proof. (1) is syntactic; (2) is the standard RO/PRF pre-image argument. ■

5.6 Theorem 11 (Mutual Decryptability and Transaction-Graph Privacy)

Statement. For every accepted spend, *both* counterparties can name the other as a *registered* Identity, while no third party can name either or link the mint to the spend.

Recipient names the issuer.

- *A1 (public)*: the note carries m_{iss} and an EUF-CMA Schnorr; the recipient reads M_{iss} directly.
- *A2 (private)*: the recipient decrypts $eIss$ under k to M_{iss} (Theorem 7.1). Theorem 8(5) certifies it registered, and Theorem 12 makes it the minter's own – the name is right, not merely registered.

Issuer names the depositor.

- *A1, A2 (addressed)*: the issuer chose pk_{recv} and knows M_{rec} . The mailbox leaf $\text{Poseidon}(\tau_M, M_{\text{rec}}, pk_{\text{recv}}, s')$, checked against a posted root before paying, tells it the key belongs to that Identity, and Theorem 8(3)-(4) make that Identity's accounts the only ones that spend.
- *B1 (bearer)*: the issuer decrypts $eDepForIss$ from **SpentCoupledB1** under sk_{iss} to M_{dep} (Theorem 9), certified registered.

Both directions resolve to a registered Identity point and – via the off-chain `identity_data` path (Theorem 6) – the real-world KYC identity. This is the BUCK mutual-decryptability invariant, preserved across the pool detour and *enforced*: the membership relations make an un-nameable counterparty *un-spendable*, so the invariant holds against a colluding pair, not merely an honest one.

Privacy corollary (what Mallory cannot do). The chain shows an opaque cm , a fresh nf , a public (*face, recipient*), and ciphertexts and commitments that are IND-CPA or uniform: $eIss$, Q, U, T at mint; $eEnc$, $eDepForIss$ and the perfectly hiding P_{dep} at spend. The published subtrees' leaves are salted. Lacking any counterparty secret, Mallory learns no Identity, no $cm \leftrightarrow nf$ link (Theorem 10), and no link between two notes to one recipient (Theorem 7.3). Bilateral disclosure between the two parties is preserved; bulk harvest is defeated. ■

5.7 Theorem 12 (The Key Tie – A2 Naming)

Theorem 7’s remark leaves one freedom: $eIss$ is bound to the minter’s own Identity under a key pk_Q the minter chose. A minter holding the recipient’s pk_{recv} and a registered sock puppet M_B sets $pk_Q = pk_{\text{recv}} + r'^{-1}(M_B - M_{\text{iss}})$; then $eIss = (r'G, M_{\text{iss}} + r'pk_Q) = (r'G, M_B + r'pk_{\text{recv}})$. L1-L5 hold, the recipient decrypts a registered M_B , (5) holds, and an honest recipient would name the puppet. Relation (6) is what refuses it.

Statement. Suppose an A2 note was minted under an accepted `verifyIssuerReenc` (Theorem 7.2) and deposited under an accepted fold (Theorem 8). Then under DLog, with H hashed to the curve, the Identity the fold certifies is the minter’s: $M_I = M_{\text{iss}}$. A receipt that checks

$$M_{\text{named}} = C_i - T + \gamma H$$

names the same point.

Proof. Theorem 7’s extractor gives $R_i = r'G$, $T = r'pk_Q + \gamma H$ and $C_i = M_{\text{iss}} + r'pk_Q$. Theorem 8’s gives, over the same committed $(eIss, T)$ (clause 0), the same r' – discrete logarithms in a prime-order group are unique – with $C_i = M_I + r'kG$ and $T = r'kG + \gamma_m H$. Subtracting, $M_{\text{iss}} - M_I = r'(kG - pk_Q) = (\gamma - \gamma_m)H$. Both points are registered, so a reduction that issued or extracted every registered scalar knows both logarithms, and $M_I \neq M_{\text{iss}}$ yields $\log_G H$. Hence $M_I = M_{\text{iss}}$, then $\gamma_m = \gamma$ and $pk_Q = kG$, and $C_i - T + \gamma H = M_{\text{iss}} + r'pk_Q - r'pk_Q = M_{\text{iss}}$. ■

Implications.

- **The split is refused where it matters.** At spend, by the fold, because the chain cannot look anything up there: cm and $idHash$ are hidden and the nullifier reveals nothing, so the only channel from mint to spend is $idHash$, which is why it commits T . A receipt repeats the check off chain with the γ the delivery carried.
- **Why the blind is on H .** Blinded on G , (6) binds nothing: γ absorbs any difference of Identities. A prime-order group has no rearrangement that repairs a known-logarithm blind.
- **Whose Identity, and whether registered, are separate facts.** Relation (5) says the decrypted point is REGISTERED; the tie says it is the MINTER’S. Registration cannot supply the second, since `register` deliberately refuses a caller-supplied leaf and so ties no credential’s plaintext to any accumulator leaf. Both relations are needed.

5.8 Sanity Check (py_ecc on BN254)

```
# Reuse ORDER, G1, multiply, add, neg, eq, rand, hashlib and m from Parts I-II.
# Alice (m_iss = m) mints to Bob, whose receiving secret k is independent of
# his identity scalar m_rec. H and the leaf functions are the protocol’s own.
from alberta_buck.wallet.nums import H_PEDERSEN as H
from alberta_buck.wallet.poseidon import poseidon
from alberta_buck.wallet.elgamal import ElGamalCiphertext
from alberta_buck.wallet.notes import TAG_NULLIFIER, id_hash_a2, nullifier
from alberta_buck.registry.tree import receiving_leaf, identity_leaf_salted

def enc(P, pk, r): return (multiply(G1, r), add(P, multiply(pk, r)))
def dec(R, C, sk): return add(C, neg(multiply(R, sk)))
def sub(P, Q):      return add(P, neg(Q))
def ok(what):       print(f"{what + ':'<58} PASS")

m_iss = m; M_iss = multiply(G1, m_iss)
m_rec = int(hashlib.sha256(b"Bob | Alberta | AIC-2026").hexdigest(), 16) % ORDER
M_rec = multiply(G1, m_rec)
k = rand(); pk_recv = multiply(G1, k)          # Bob’s mailbox
s_rec, s_iss = rand(), rand()                # wallet-derived salts
```

```

certified = {receiving_leaf(m_rec, k, s_rec),          # Bob: (m_rec, k)
             identity_leaf_salted(M_iss, s_iss)}      # Alice's naming leaf

# --- Theorem 7: k opens every addressed ciphertext ---
v, rn, rp, gamma = 25, rand(), rand(), rand()
eNote, eIss = enc(multiply(G1, v), pk_recv, rn), enc(M_iss, pk_recv, rp)
T = add(multiply(pk_recv, rp), multiply(H, gamma))    # the binding's T
assert eq(dec(*eNote, k), multiply(G1, v)) and eq(dec(*eIss, k), M_iss)
ok("Theorem 7 (k decrypts eNote and eIss)")

# --- Theorem 8: a thief with the stolen k and its own Identity fails (4) ---
m_thief = rand()
assert receiving_leaf(m_rec, k, s_rec) in certified
assert receiving_leaf(m_thief, k, s_rec) not in certified
ok("Theorem 8 (stolen k, thief's Identity: no certified leaf)")

# --- Theorem 8(5): a note keyed to a throwaway point names no one ---
eIss_bad = enc(multiply(G1, rand()), pk_recv, rp)
assert identity_leaf_salted(dec(*eIss_bad, k), s_iss) not in certified
ok("Theorem 8 (A2 throwaway issuer: (5) has no witness)")

# --- Theorem 9: with a known-log generator P_dep opens to anyone ---
h = rand(); Hk = multiply(G1, h); b = rand()
m_other = rand()                                # a disclosed scalar
P_dep = add(multiply(G1, m_rec), multiply(Hk, b))
b_shift = (b + (m_rec - m_other) * pow(h, -1, ORDER)) % ORDER
assert eq(P_dep, add(multiply(G1, m_other), multiply(Hk, b_shift)))
ok("Theorem 9 (known-log H: the shift works, so H is hashed)")

# --- Theorem 10: one flavor-agnostic nullifier, deterministic ---
rho = rand()
idHash = id_hash_a2(ElGamalCiphertext(*eNote), ElGamalCiphertext(*eIss), T)
assert nullifier(rho, idHash) == nullifier(rho, idHash) == poseidon([TAG_NULLIFIER, rho, idHash])
ok("Theorem 10 (nullifier determinism)")

# --- Theorem 11: B1's issuer names the depositor ---
sk_iss = rand(); pk_iss = multiply(G1, sk_iss); rd = rand()
assert eq(dec(*enc(M_rec, pk_iss, rd), sk_iss), M_rec)
ok("Theorem 11 (B1 issuer decrypts the depositor)")

# --- Theorem 12: the key split, and the tie that refuses it ---
M_B = multiply(G1, rand())                      # the minter's sock puppet
pk_Q = add(pk_recv, multiply(sub(M_B, M_iss), pow(rp, -1, ORDER)))
eIss_split = enc(M_iss, pk_Q, rp)                # L1-L5 hold under pk_Q
assert eq(dec(*eIss_split, k), M_B)              # ...Bob's k opens the puppet
T_split = add(multiply(pk_Q, rp), multiply(H, gamma))
tie = add(multiply(G1, rp * k % ORDER), multiply(H, gamma))
assert eq(sub(T_split, tie), sub(M_B, M_iss))    # (6) needs M_B - M_iss = delta*H
ok("Theorem 12 (split: the tie needs a log to H)")

assert eq(T, tie) and eq(add(sub(eIss[1], T), multiply(H, gamma)), M_iss)
ok("Theorem 12 (honest: tie holds, C_i - T + gamma*H = M_iss)")

Theorem 7 (k decrypts eNote and eIss):          PASS
Theorem 8 (stolen k, thief's Identity: no certified leaf): PASS
Theorem 8 (A2 throwaway issuer: (5) has no witness): PASS
Theorem 9 (known-log H: the shift works, so H is hashed): PASS
Theorem 10 (nullifier determinism):             PASS
Theorem 11 (B1 issuer decrypts the depositor):  PASS
Theorem 12 (split: the tie needs a log to H):   PASS
Theorem 12 (honest: tie holds, C_i - T + gamma*H = M_iss): PASS

```

How to read. Theorem 7: the receiving secret, not the identity scalar, opens the note. Theorem 8: the stolen key does not help a thief whose own Identity was never paired with it, and a note keyed to a throwaway point decrypts to an Identity no one certified. Theorem 9: with a generator

whose logarithm is known, the committed point opens to any disclosed Identity, which is why H is hashed to the curve. Theorem 10: one deterministic nullifier. Theorem 11: the bearer issuer names its depositor. Theorem 12: the split mint decrypts under Bob’s key to the puppet, and the tie then demands exactly $M_B - M_{iss}$ as a multiple of H – a discrete logarithm – while an honest mint satisfies it and the receipt’s check recovers the minter.

5.9 Worked Example: An A2 Spend End to End

Theorems 7-12 in one A2 transfer: Alice issues privately to Bob, who deposits from an account no one could have pinned at mint. Executable reference: `alberta_buck/wallet/unilateral_a2.py`, `delivery.py` and `deposit_fold.py`, with `test_unilateral_a2.py`, `test_delivery.py` and `test_receiving_key.py` and on chain `NotesA2Tie.t.sol` and `NotesE2E.t.sol`. The relation trace:

1. **Register** (Parts I-III). Alice and Bob each hold a PS-certified Identity (m, M) and registered accounts. Bob’s registry certifies $\ell_R(m_{rec}, k, s)$, his Identity paired with his receiving key; Alice’s certifies a naming leaf $\ell_S(M_{iss}, s_{iss})$.
2. **Mint** (Theorem 7). Alice learns M_{rec} and pk_{recv} off chain and checks the mailbox leaf, then mints $eNote$ and $eIss$ with the binding (Q, U, T) . The leaf $cm = \text{Poseidon}_6(T_{CM}, A2, v, \rho, \text{Poseidon}_{11}(T_{ID}, e))$ is folded into `noteTree`.
3. **Deliver and check** (Theorems 11, 12). The payload $-\rho, r', \gamma, s_{iss}$ and the openings – travels wrapped to pk_{recv} . Bob decrypts $eIss$ under k to M_{iss} , checks $C_i - T + \gamma H = M_{iss}$, and rebuilds the path to $\ell_S(M_{iss}, s_{iss})$ from the published subtree before accepting the note.
4. **Deposit** (Theorems 8, 10, 12). From any registered account, Bob submits `spendCoupledA2` naming a posted `rt_id`: the spend proof (cm under `noteRoot`, nf) and one fold proof over the six relations. The pool pays `face` BUCK.
5. **Receipt** (Theorem 11). Bob’s receipt names himself and Alice, and the check of step 3 makes the naming the minter’s.

No on-chain value reveals an Identity; mint and spend are unlinkable to Mallory (Theorem 10); a throwaway $eIss$ fails step 3 and would revert at step 4 (Theorem 8(5)); and a split $eIss$ fails the tie at step 4 (Theorem 12).

5.10 Mint-Circuit Obligations

Three correctness obligations fall on the mint circuit rather than the spend. The batch mint (`circuits/mint_batch.circom`, deployed as per-N pinned variants `mint_batch_n${N}.circom`; see `alberta-buck-notes-rollup-mint.org`) discharges them.

5.10.1 Obligation 1 (Field-Overflow / Value-Conservation Range Check) – *Discharged in circuit*

Value conservation $\sum_{i=1}^N v_i == \text{totalFace}$ is an equation in $\mathbb{F}_r$ with $r \approx 2^{254}$ and therefore admits integer-overflow witnesses unless each v_i is bounded. `mint_batch.circom` applies `Num2Bits(128)` to every v_i and to `totalFace` (constraint family (R) in the circuit source). With $N \leq 2^{20}$ (the tree-depth cap) and 128-bit per-leaf face, the integer sum cannot overflow: $N \cdot 2^{128} \leq 2^{148} \ll 2^{254} = r$. No spend can release more BUCK than was escrowed through an integer-versus-field mismatch on the mint side.

5.10.2 Obligation 2 (Single transferFrom $\leftrightarrow$ N Commitments) – *Discharged by SNARK binding*

The `totalFace` public input of the mint SNARK is both (i) the value passed to `buck.transferFrom` and (ii) the value the circuit forces to equal $\sum_i v_i$. The single `Transfer` event of size `totalFace` is therefore bound to the Groth16 proof’s public inputs – which include the entire `cms[]` array on the per- N pinned verifiers (see alberta-buck-notes-rollup-mint.org, "Calldata delivery"). No batch-identifier storage slot is needed for soundness; analytics that want a transfer $\leftrightarrow$ batch correspondence recover it from the `Minted` event’s `cms` and the simultaneous `Transfer`.

5.10.3 Obligation 3 (Membership, Nullifier, Value) – *Split between mint and spend*

1. **Merkle membership (mint side).** `mint_batch` performs an in-circuit dual Merkle walk per leaf (constraint family (M) in the circuit source): the empty-seat walk constrains every witness sibling against the rolling prior root, and the filled-seat walk advances the rolling root with cm_i inserted. After all N leaves the final rolling root is constrained to equal the public `newRoot`, which the contract accepts into its root ring; spends prove inclusion against any root in that window.
2. **Nullifier determinism (spend side).** `circuits/spend.circom`, the one note proof every spend reuses, recomputes $nf = \text{Poseidon}_3(T_{NF}, \rho, \text{idHash})$ from the committed opening (Theorem 10); the folds recompute it too, which is how they are joined to the spend.
3. **Value conservation (spend side).** The spend circuit outputs `faceReleased` equal to the committed v , the same 128-bit-bounded quantity range-checked at mint. The mint-side range check (Obligation 1) is load-bearing here: a spend circuit that trusts v without re-bounding it relies on the mint circuit having done so.

The leaf layout is $\text{Poseidon}_6(T_{CM}, \text{flavor}, v, \rho, \text{idHash}, \text{predicate})$ over BN254 at tree depth 20, pinned by the per- N verifiers. The A2 mint additionally exposes each leaf’s `eIss` and `T` and recomputes $\text{idHash} = \text{Poseidon}_{11}(T_{ID}, eNote, eIss, T)$ in circuit, so the binding `Notes.mint` checks is the one the note commits: $7N + 4$ public signals per batch.

6 Security Assumptions Summary

Theorem	Primitive	Assumption needed
Theorem 1 (PS completeness, wallet-internal)	PS	bilinearity of e
Theorem 1' (masked completeness)	PS + Y_1	bilinearity of e ; $A \neq O$
Lemma 2 (fresh-signature distribution)	PS	prime-order G_1 , uniform t
Theorem 2' (presentation hiding)	PS + Y_1	prime-order G_1 , uniform a, b
Theorem R-HVZK (registration ZK)	masked PS + FS	uniform nonces; ROM for NIZK
Theorem R-sound (special soundness)	masked PS	none for extractor
Theorem R-comp (composition)	masked PS + CP	shared nonces
Theorem R-unforge (registration unforgeability)	masked PS + FS	PS-CM (PS 2016 Sec. 6.1, Assumption 2), ROM, forking
Theorem R-blind (issuer-blindness of registration)	masked PS + ElGamal	Theorems 2', R-HVZK; DDH on G_1
Theorem 3 (CP completeness)	CP	group axioms
Theorem 4 (CP special soundness)	CP	none for extractor; DLog for cheating-prover reduction
Theorem 5 (CP HVZK)	CP	uniform k_1, k_2
NIZK corollary	CP + FS	ROM
Theorem 6 (end-to-end)	masked PS + CP	PS-CM (PS unforgeability), DLog, ROM
Transfer mutual-decryptability (cor.)	ElGamal + CP	ElGamal correctness; Theorem 6
Theorem 7 (mint addressing + anti-framing)	ElGamal + Okamoto	group axioms; Schnorr/Okamoto soundness
Theorem 8 (folded deposit gate)	one SNARK over one witness	SNARK knowledge soundness; Poseidon CR; DLog (A2 key)
Theorem 9 (depositor-binding soundness)	Okamoto + membership + Schnorr	SNARK knowledge soundness, Schnorr EUF-CMA; DLog with H_{PED} hashed to the curve
Theorem 10 (nullifier + graph privacy)	Poseidon-PRF	ROM (Poseidon pre-image)
Theorem 11 (mutual decryptability)	ElGamal	ElGamal correctness; Schnorr EUF-CMA
Theorem 12 (A2 key tie)	Okamoto + SNARK	Theorems 7, 8; DLog with H_{PED} hashed to the curve

DLog, DDH, the ROM and SNARK knowledge soundness are already load-bearing for Ethereum itself (ECDSA on secp256k1 assumes DLog; every pairing-based SNARK verifier on L1 uses the same ROM reductions). PS-CM is not: it is the credential scheme's own assumption, in the committed-message key form that publishing Y_1 requires, and adopting it is a decision recorded in the A' plan. The earlier statement that Alberta Buck introduces no novel cryptographic assumption is withdrawn.

7 Implementation Notes

7.1 What the Contract Must Enforce

The theorems assume a well-formed verifier. Four conditions on the on-chain code are load-bearing:

1. **Registration: the guards of Part I.** $A \neq O$ is critical (at $A = O$ the masked equation holds for every m : the trivial presentation attack, the Identity doc's Attack 9 / Link 1 with $a = 0$ in place of $t = 0$); $pk \neq O$ and $R \neq O$ are critical (no key ownership; $C = mG$ published); $B \neq O$ is defensive; the Y_1 consistency check at `trustIssuer` is what Theorem R-sound's extraction needs. Part I's "What the Verifier Checks, and Why" gives each with its revert string.
2. **Registration: G_2 subgroup check.** The `ecPairing` precompile (post-EIP-197) validates that X, Y lie in the correct prime-order subgroup of G_2 . Without this, small-subgroup attacks on the twist curve could forge pairings. Theorem 1 is stated over the correct subgroup; enforcement is the precompile's responsibility.
3. **Verification: E_{alice} read from storage, not calldata.** Theorem 6's reduction routes through the registration-time binding of E_{alice} to issuer-certified m . If Alice supplies E_{alice} via calldata the binding is severed and Theorem 6 does not apply.
4. **Notes: every fold input derived on chain.** Theorem 8's clause (3) is about a registered account only because the adapter reads pk_{dep} and E_{dep} from the registry at `msg.sender` and derives nf , the face and $eEnc$'s limbs itself; the identity root must be one the registry accepts for the Notes consumer; and the B1 membership proof must receive the same P_{dep} the sigma verified. A caller-supplied copy of any of them severs the clause it feeds.

7.2 What the Prover Must Enforce

- **Uniform, fresh randomness for $a, b, m_t, b_t, r_t, sk_t, k_1, k_2, r_b$.** Theorems 2', R-HVZK and 5 depend on uniform distributions, and Theorem 2' additionally on b never being reused across accounts. Proposition R-fresh lists the explicit breaks for each reused value; RFC 6979-style derivation of the four proof nonces is the recommended hardening. A biased RNG converts the hiding into a detectable correlation and converts perfect HVZK into approximate HVZK with an adversarial advantage proportional to the bias.
- **Never publish σ or a rerandomization of it.** Lemma 2's remark: any such pair is a candidate test on m . Backups, receipts and demos must carry the presentation or nothing.
- **Fiat-Shamir domain separation.** The hash input must include all public inputs and context (`msg.sender, spender, chainid, registry`) and end in the protocol's v2 domain word. Omitting any one opens a cross-context replay path; the forking lemma reduction is unaffected, but the NIZK is no longer bound to the intended transaction, or to the intended protocol.

7.3 Scope Not Covered Here

- **Knowledge soundness** of the registration-time NIZK that binds σ' to E_{alice} : this is a standard Schnorr-family sigma protocol over a linear pairing relation (see alberta-buck-identity.org, "One-Time (PS Signature + NIZK Verification at Registration)") and follows the same completeness / soundness / ZK template as Theorems 3-5. A separate write-up at the same level of detail is the natural next step.

- *Circuit soundness* of the BUCK Notes spend: Part IV proves that the folded deposit gates (A1/A2), the depositor-binding sigma (B1) and its membership proof imply the desired guarantees *assuming* knowledge soundness of the underlying SNARK system, and that each circuit carries the relations its clear-text reference states. Soundness of the *proof system itself* (Groth16 / PLONK / STARK) depends on the chosen proof system’s trusted setup (if any) and circuit correctness; this is orthogonal to the contents of this document.
- *Collision-resistance and hiding* of the Poseidon commitment hash H_{cm} and pseudorandomness of the Poseidon nullifier PRF. Poseidon is modelled as a random oracle in the SNARK circuit for the double-spend and commitment-binding arguments; its concrete security on BN254 has been independently studied and is assumed here. See the Poseidon-hash cryptanalysis literature for current best attacks and parameter choices.

7.4 The Long-Horizon Threat: Harvest-Now, Decrypt-Later

Every unlinkability guarantee in this document that rests on *ElGamal* – cross-account unlinkability, the *issuer-blindness* of the identity layer (a compelled or breached issuer cannot map its issued identities to chain accounts), and the third-party non-disclosure of A2 Notes – is *computational*, reducing to DDH (hence DLog) on $\mathbb{G}_1$. Only Theorem 2’ (the hiding of the presentation) is information-theoretic and survives unconditionally; the pre-A’ published pair was candidate-testable classically, by anyone holding m , with no break at all. This asymmetry has a sharp consequence against a powerful, patient adversary (a state or military actor with archival access).

The on-chain credentials $E_{\text{addr}} = (rG, M + rpk)$ are permanent public history. An adversary who learns the *plaintext* identity points $\{M_i\}$ – paradigmatically a **compelled or breached issuer**, whose KYC records *are* exactly that set – cannot link them to chain accounts *today*: testing whether a given E encrypts a known M_i is a plaintext-checking attack, i.e. deciding whether $(G, pk, R, C - M_i)$ is a DDH tuple, infeasible under DDH (this is the load-bearing observation behind the Identity doc’s privacy-boundary table and the paper’s *Issuer-blindness* theorem). But a future discrete-log break – in particular a cryptographically relevant quantum computer running Shor against $\mathbb{G}_1$ – collapses that DDH decision into a cheap test, retroactively de-anonymising *all past* identity and A2-Note traffic from the harvested $\{M_i\}$ plus the archived chain. The presentation’s hiding is unaffected, but it does not hide the $M \leftrightarrow E_{\text{addr}}$ relation; only the ElGamal layer does, and only computationally.

Two points are worth recording for the threat model:

1. **Revocation does not bound it.** Clearing a holder from its registry’s subtree ends its liveness, but the harvest target here is the persistent $M \leftrightarrow E_{\text{addr}}$ link, which a quantum DLog break recovers wholesale, revoked or not.
2. **The migration is encryption-layer, not pairing-layer.** The BLS12-381 path (EIP-2537, "alt_bn128 Security Horizon" in the Identity doc) hardens *pairing* security for PS verification; it does nothing for the ElGamal confidentiality that DDH underwrites. Post-quantum unlinkability requires replacing the $\mathbb{G}_1$ ElGamal credential with a post-quantum encryption (a lattice-based KEM/commitment, with a matching re-encryption-equality argument to preserve the **approve** handshake) – a substantial identity-layer redesign, and the right place to invest if the threat model includes a quantum adversary with archival access to the issuer’s records.

This is a property of *every* DDH/DLog-based confidential-payment system, not of Alberta Buck

specifically; it is recorded here because the issuer-breach corpus makes the harvest concrete and names the one party whose compromise hands an adversary the full plaintext set $\{M_i\}$.

DRAFT

8 Review Status and Remaining Work for Part I

Part I is implemented and argued but not independently reviewed. This section is the reviewer's brief and the list of what would strengthen the argument; it is kept in this document so that the proofs and the obligations on them travel together.

8.1 What the Reviewer Is Asked to Check

In the order that matters.

1. **The assumption.** Confirm that the key form $(G, Y_1; g_2, X, Y)$ with plain signing is the Section 6.1 scheme of Pointcheval and Sanders (2016) with $t = 0$, and that its EUF-CMA proof under that paper's Assumption 2 applies; say whether a non-interactive assumption from the 2018 reassessment covers it.
2. **Theorem 2'.** The bijection argument and the $1/q$ accounting; that returning P to the HIDE adversary makes the game at least as strong as the real transcript.
3. **Theorem R-HVZK.** That the four verification equations determine the four commitments from the responses, and the programming-abort bound.
4. **Theorem R-sound.** The division step; that $Y_1 = yG$ is the only place Y_1 enters; the shared-nonce composition (Theorem R-comp).
5. **Theorem R-unforge.** The load-bearing reduction. The simulation of presentations from a signature on an unrelated message, the statistical-distance accounting, the treatment of m_d , and the forking-lemma application (which query index is forked; that the commitments are fixed before the challenge).
6. **Theorem R-blind.** The hybrid order and that the final hybrid is exactly ElGamal plaintext-checking under a fresh key.
7. **Code against specification.** `_verifyCredential` (guards, canonical scalars, revert strings), `_fsRegister` (word order and count against `nizk.py`), `_checkPresentation` (the three pairs, the negations, the $-eB$ term), `trustIssuer` ($Y_1 \neq O$ and the consistency pairing). The presence of the $pk \neq O$, $R \neq O$ and nonzero-leaf rejections in the Solidity path was confirmed while writing Part I; the reviewer should confirm it again against the commit under review.
8. **Ports.** `core/rust/buck-identity/src/nizk.rs` and `core/js/src/identity-core.js` against `nizk.py`; the kernel is what wallets run. Vectors: `test/vectors/identity.json` and `core/vectors/identity`, both schema 2.
9. **Tests.** `make nix-test` (Foundry: the rejection tests in `test/IdentityRegistry.t.sol` for zero A , zero B , a tampered s_b , a swapped presentation, a zero and an inconsistent Y_1 , and "presentation is not a signature"); `alberta_buck/test/review/test_wallet_failures.py` tests 01, 02, 10, 11, 12 on both backends; `make nix-core-test-rust`, `nix-core-test-py`, `nix-core-test-js`; the two-sided witness in `alberta-buck-identity-example.org` (make `nix-venv-doc-identity-example`).
10. **Randomness.** `alberta_buck/wallet/bn254.py:rand_scalar` and the JS and wasm samplers, against Proposition R-fresh.

11. **The papers.** That the conditional phrasing in this Part, in `alberta-buck-paper.org` (issuer-blindness, non-frameability, the read/write separation) and in `alberta-buck-identity.org` matches what Part I proves and no more.

8.2 What Would Strengthen the Argument

- **Independent review** of items 1–6 by a cryptographer who did not write them; this is the gate on any production registration.
- **A settled assumption.** Item 1 decides whether Part I rests on an interactive assumption (PS 2016, Assumption 2) or a non-interactive one (PS 2018); the summary table should then name it.
- **Deterministic nonces.** RFC 6979-style derivation of m_t, b_t, r_t, sk_t from the witness and the transcript removes the RNG from the trusted base for the proof; a and b remain fresh per registration. A wallet change with new vectors.
- **Tighter or mechanized checks.** The sanity block exercises each step algebraically on random instances; a proof assistant or a symbolic check of the reduction would replace "on random instances" by "for all".
- **The two open findings.** R5 (identity-derived Note receiving keys) and R6 (issuance, policy and accumulator metadata) are outside Part I and remain to be designed; nothing in Part I is made worse or better by them.